SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

삼성청년 SW·AI아카데미

관통PJT Overview

Confidential

<알림>

본 강의는 삼성청년SW·AI아카데미의 컨텐츠로
보안서약서에 의거하여
강의 내용을 어떠한 사유로도 임의로
복사, 촬영, 녹음, 복제, 보관, 전송하거나
허가 받지 않은 저장매체를 이용한 보관,
제3자에게 누설, 공개,
또는 사용하는 등의 행위를 금합니다.

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

관통 프로젝트 Overview

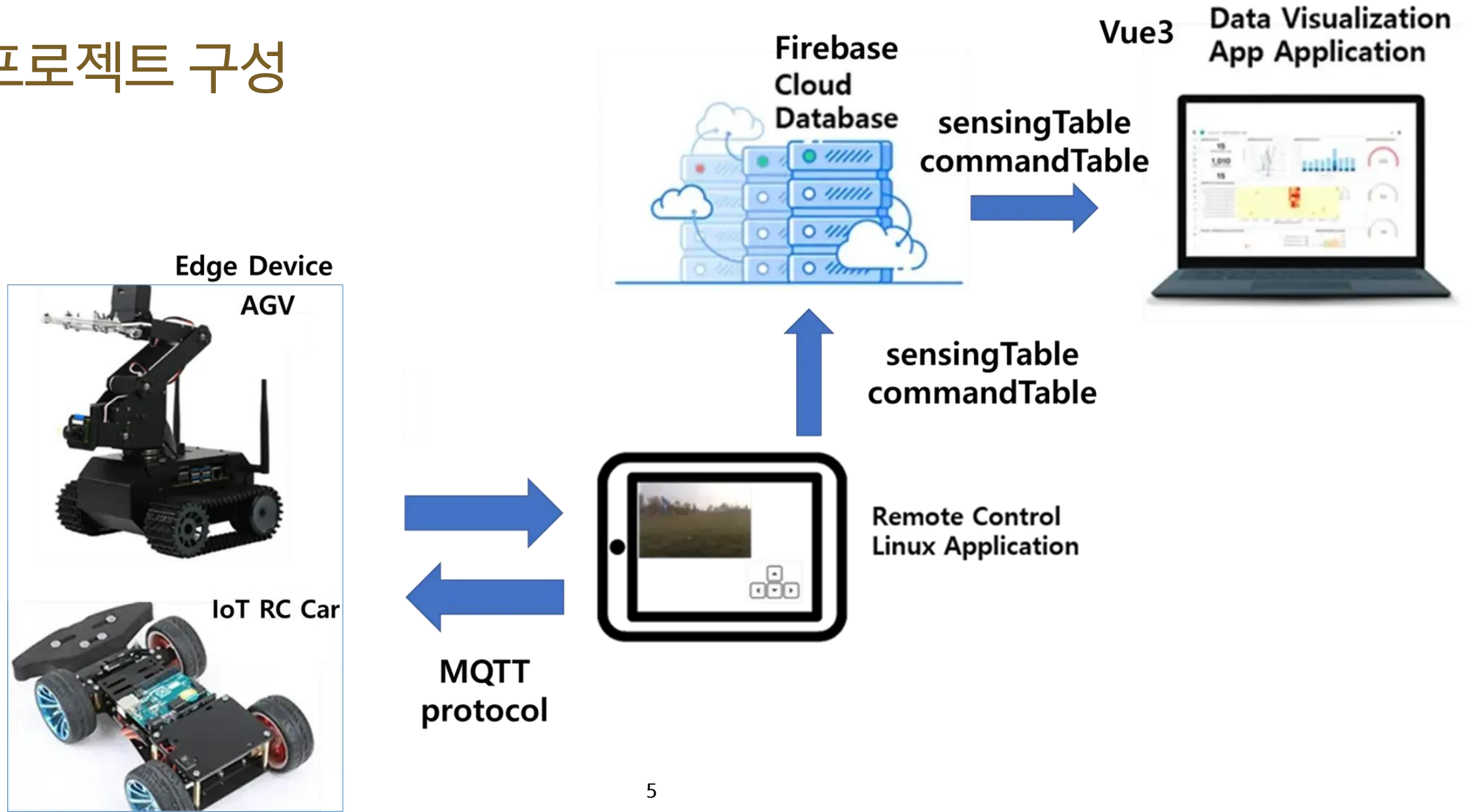
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

프로젝트 모듈별 소개

Confidential

전체 프로젝트 구성



Confidential

1. Edge Device

- 인공지능과 Computer Vision을 이용해 움직이는 AGV와 센서들을 장착한 IoT RC Car
- Python 기반

- AGV : Jetson Nano 기반, 로봇팔, 카메라 부착
  - JetPack : v4.5 (JetBot : v0.4.0)
  - TensorRT : 7.1.3
  - cuDNN : cuDNN8.0
  - Python : 3.6.9

- IoT RC Car : Raspberry Py4 기반, 다양한 센서 부착 가능
  - Raspberry Pi4 + Raspberry Pi zero
  - Python 3.11.2

Edge Device



Confidential

2. GUI 원격제어기

- Raspberry Pi5 + Qt 로 제작
- 사용자의 입력에 따라 Edge Device를 원격으로 제어
- Edge Device 모니터링 기능

Qt

Raspberry Pi®



Remote Control
Linux Application

Confidential

3. Cloud Database

- GUI 원격제어기로부터 받은 Data를 저장
- Firebase 기반 Cloud Database 를 운영 (서버리스)

Firebase



Confidential

4. Web Visualization

- Cloud Database에 저장된 DB의 값을 시각화하는 웹사이트
- Node.js 기반 Local 웹 서버
- Vue.js 와 Chart.js 를 사용하여 데이터 시각화



Confidential

5. AI 기능 추가

프로젝트에 AI를 함께 사용한다.

다양한 개발 시나리오를 사용한다.

- Vision
- LLM
- AI Agent
- On-device AI

최종 결과물

Confidential

Web + 임베디드 + AI 를 모두 장착한

AI 임베디드 PJT 를 완성한다.

관통PJT 운영 방식 소개

Confidential

관통PJT 는 페어 프로그래밍 활동이다.
임베디드 트랙의 관통PJT는 2개의 프로젝트 주제로 진행된다.

1. AGV
2. IoT RC Car

SW개발자로서 가장 중요한 덕목은 팀워크다.
협업이 잘되는 팀일 수록 더 성공적인 PJT를 구현할 수 있게 된다.

공통 진행 내용 세팅

Confidential

챕터의 포인트

- GUI 원격 제어기 소개
- UI 제작하기
- commandData 생성하기
- commandData 기록하기

GUI 원격 제어기 소개

Confidential

목표

- Edge Device를 원격 제어할 수 있는 GUI 원격 제어기를 만든다.
- Edge Device로부터 sensing Data로 받아서 화면에 표시한다.

GUI 원격 제어기 소개

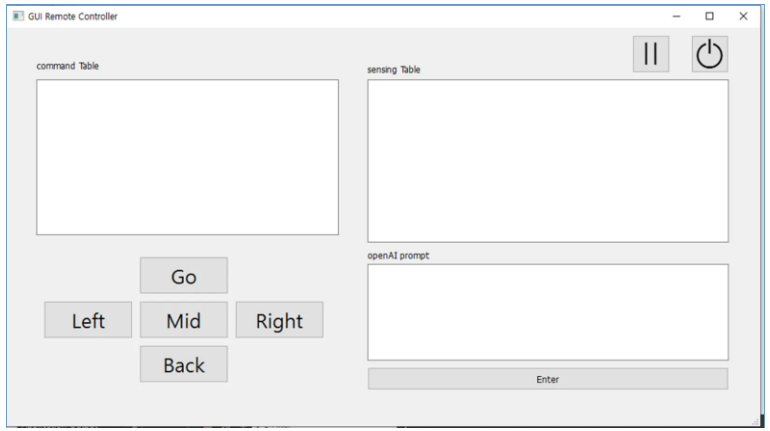
Confidential

GUI 원격 제어기는 Edge Device를 원격 제어하거나
Edge Device로부터 받은 Data를 DB로 전송하는 역할을 한다.

- **Qt로 제작**
- **sensing Data / command Data 모니터링**



MQTT

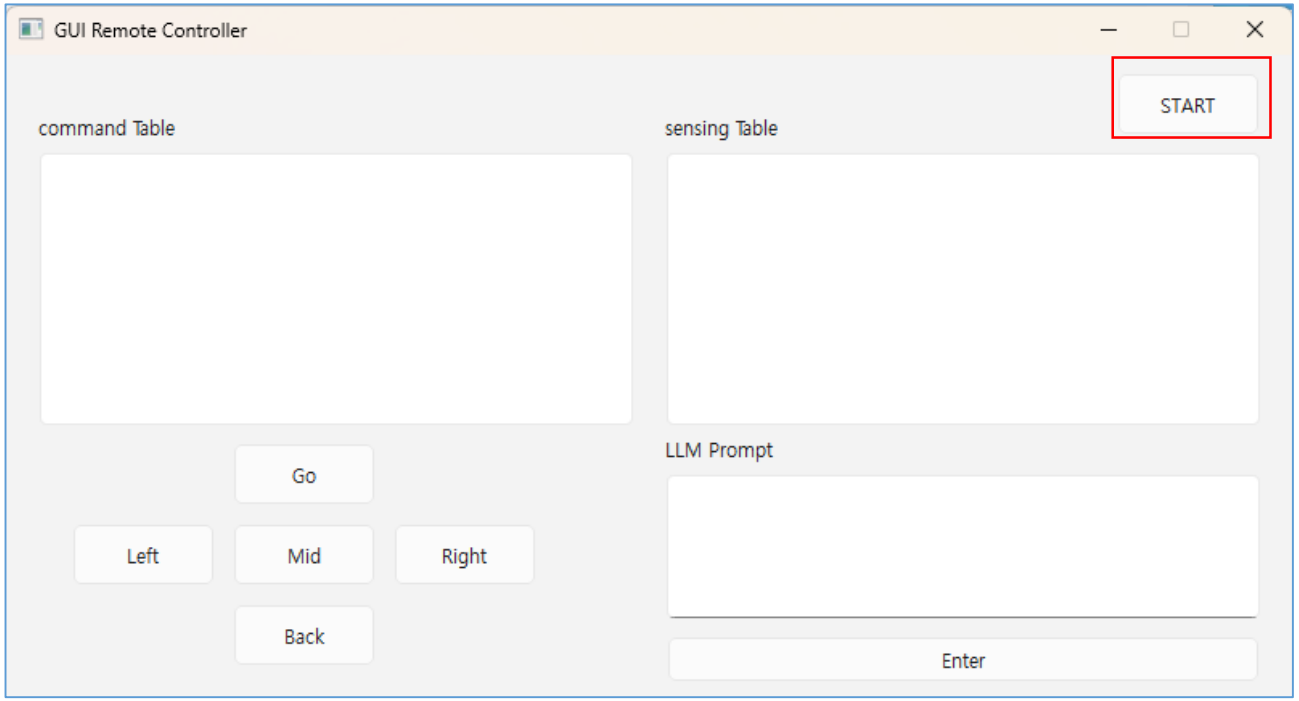


Firebase

Confidential

전원 버튼

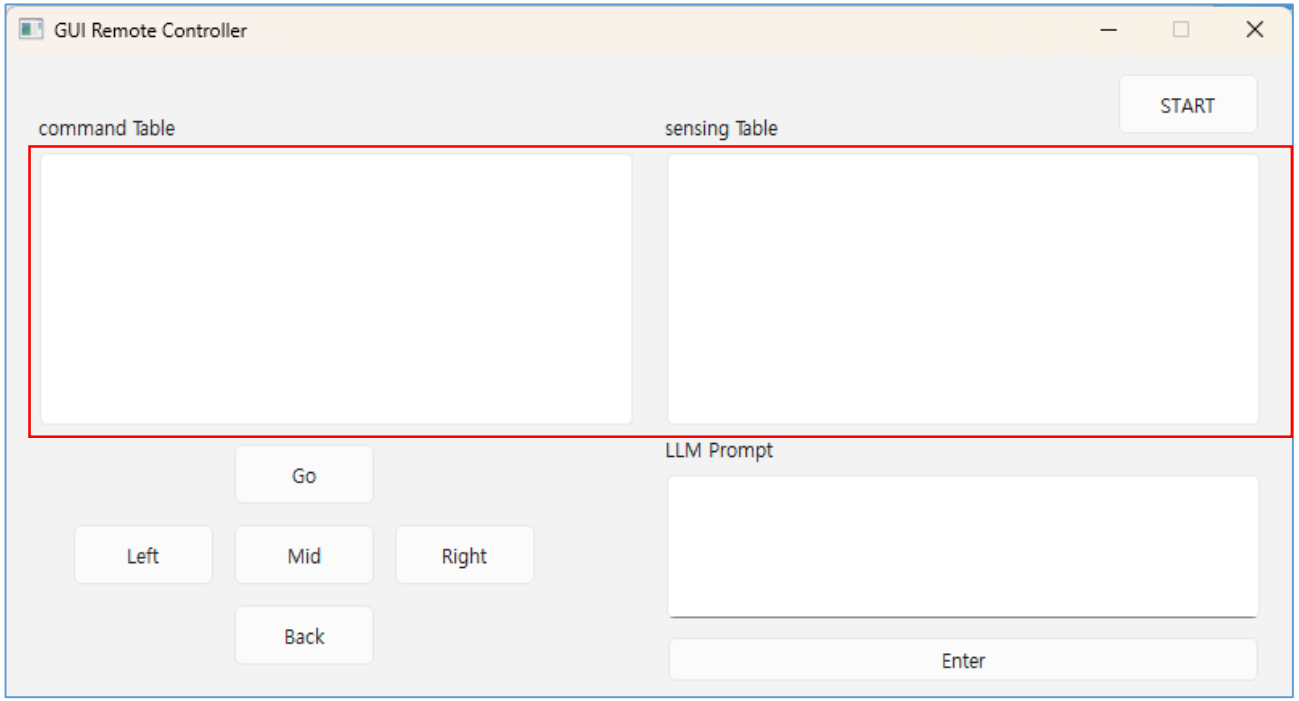
- 시작 버튼 : 비동기적으로 실시간 모니터링 기능과 원격 제어 기능 동작



Confidential

모니터링 화면

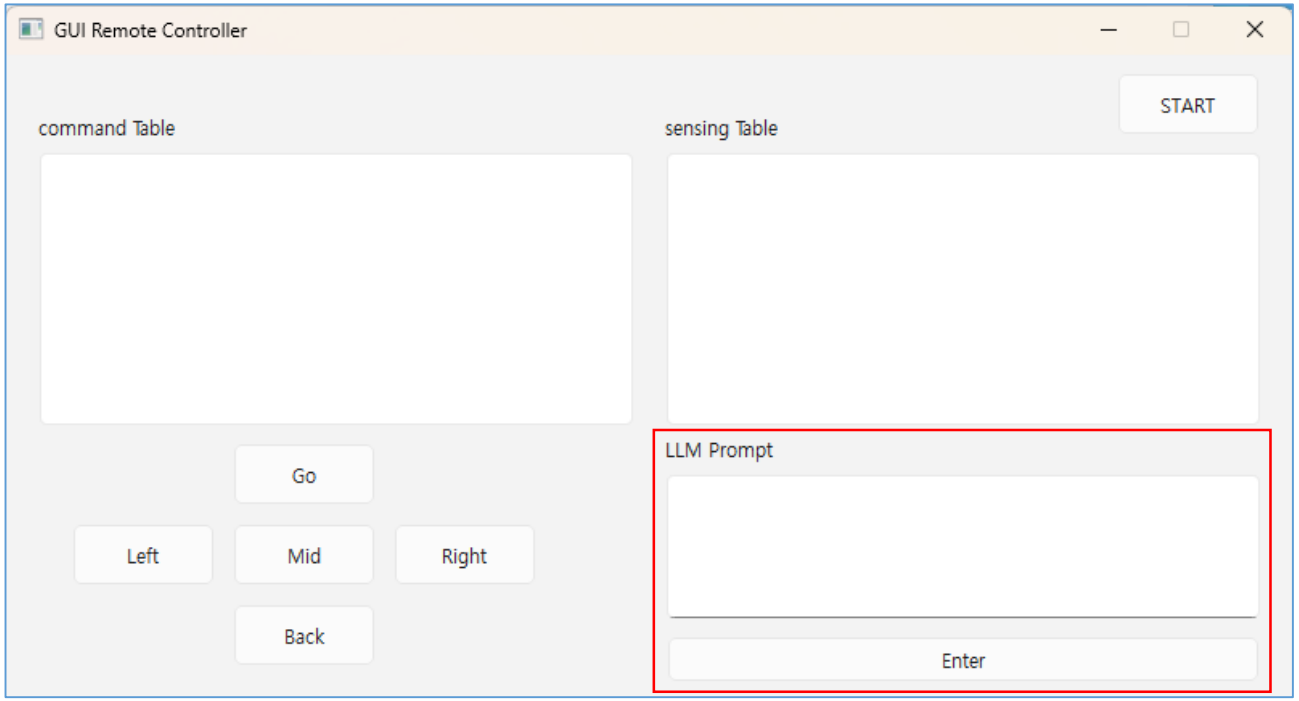
- command Table : user가 누른 원격 제어버튼 명령이 시간 정보와 함께 표시
- sensing Table : Edge Device에서 넘어온 Data가 시간 정보와 함께 표시
- 비동기 동작
- 가장 최신 15개 데이터만 출력



Confidential

LLM prompt 화면

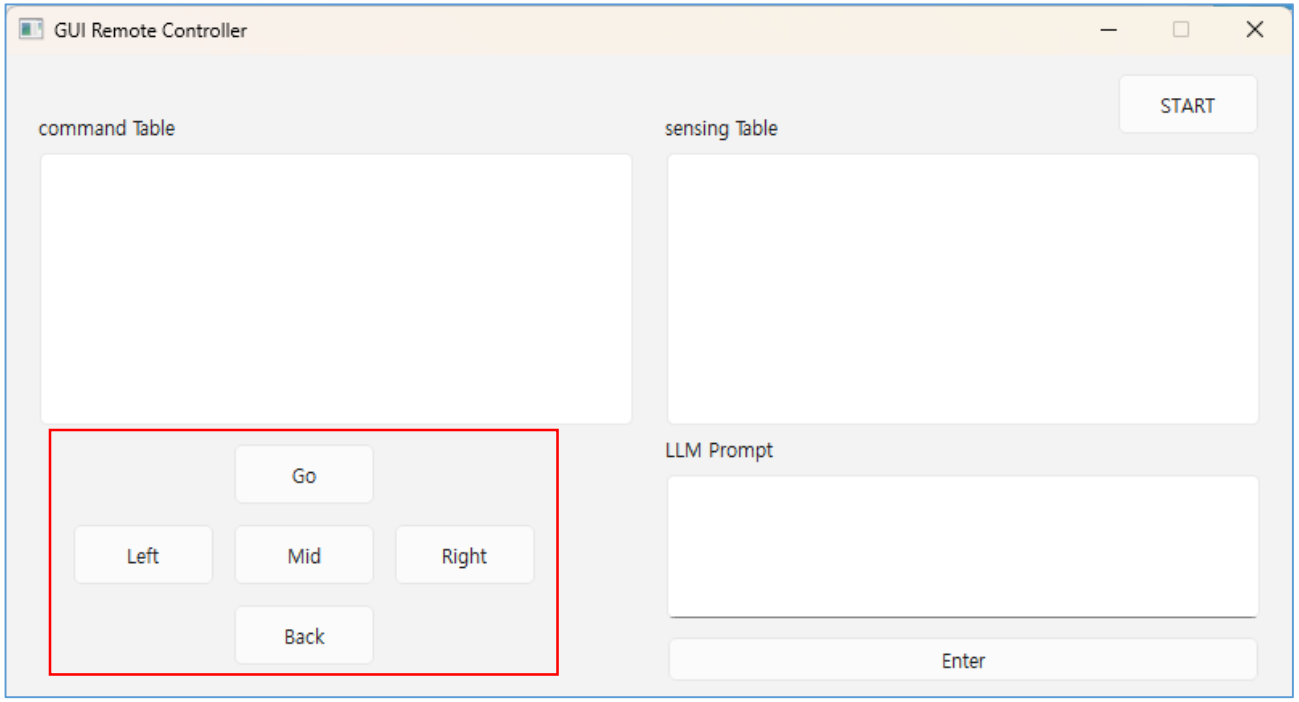
- User가 LLM에게 보낼 명령을 받는다.
- LLM은 User의 입력 프롬프트에 맞춰 결과를 출력한다.



Confidential

버튼 제어부

- User가 버튼을 눌러 Edge Device로 명령 신호를 보낼 수 있다.
- 명령은 시간 정보와 함께, command Table에 표시된다.



SAMSUNG SW AI ACADEMY FOR YOUTH

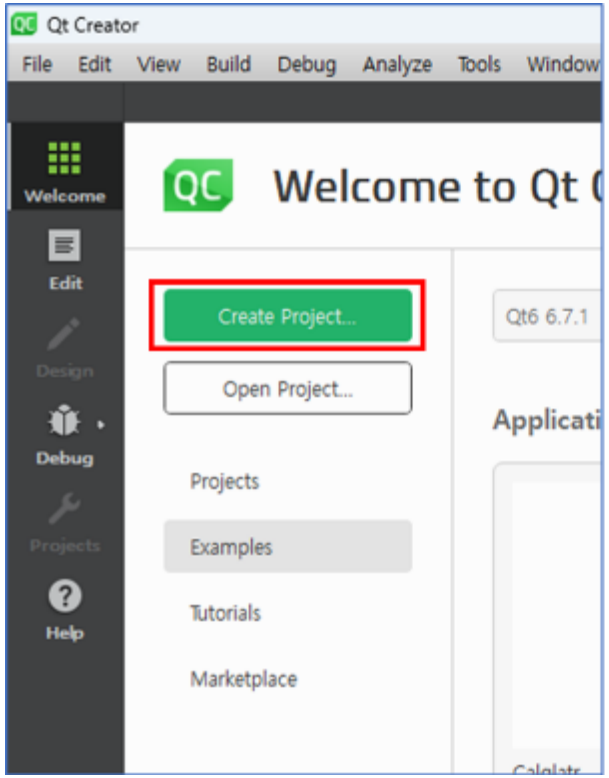
함께가요 미래로!
Enabling People

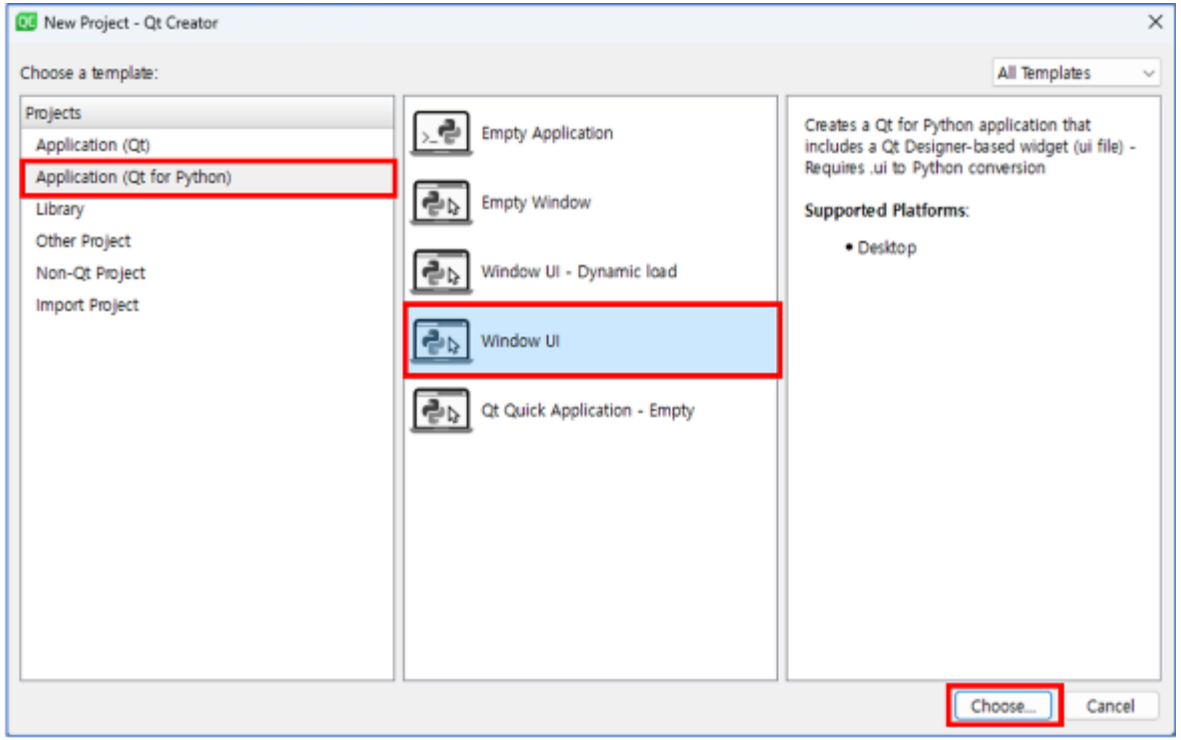
UI 제작하기

Confidential

QT Creator 실행

- 파일 → 새 프로젝트 클릭
- Create Project → Application(Qt for Python) → Window UI → Choose



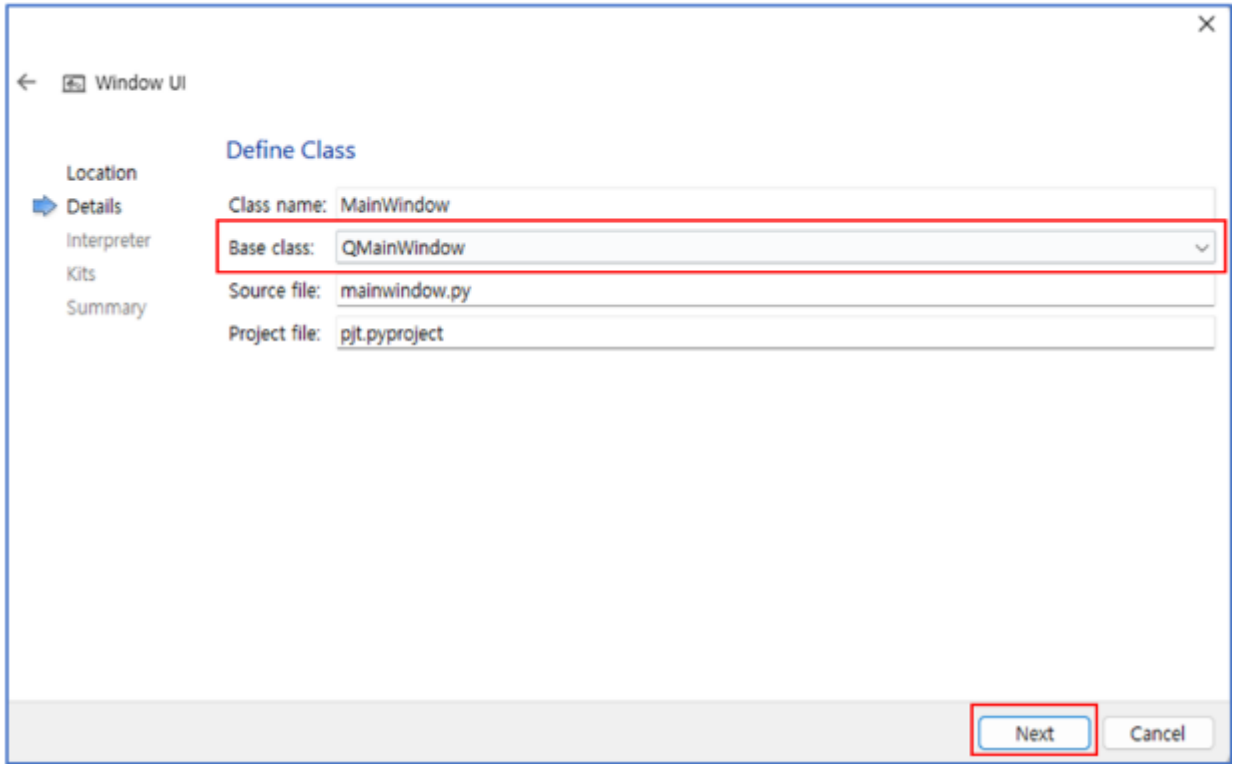


QMainWindow 로 생성

- **Base Class : QMainWindow 변경**
- **Next**

이제부터 우린 항상 QMainWindow 로 제작한다.

- **Rpi5 와의 연동을 위한 빌드업**



가상 환경 설정

가상 환경을 설정한다.

- 터미널 창을 열고(5 Terminal), 가상 환경을 활성화 한다.

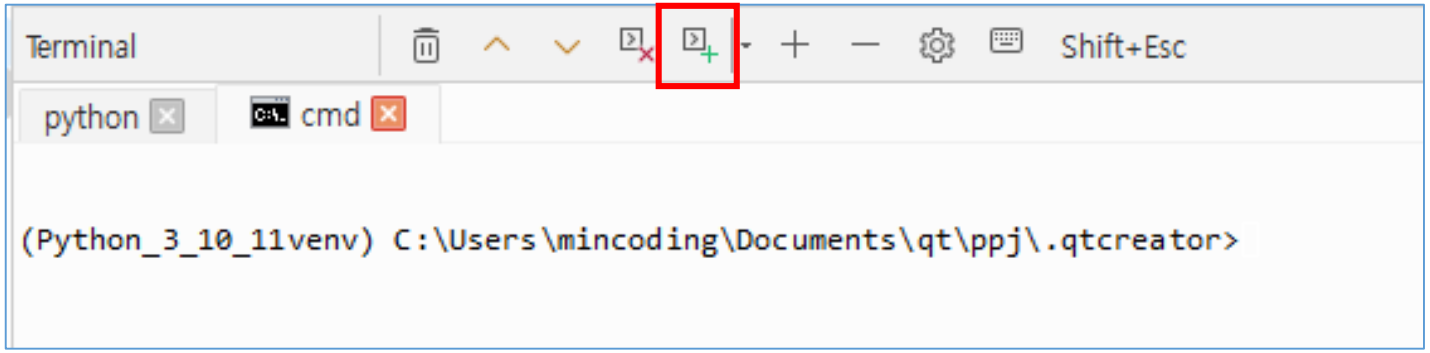
>cd .qtcreator

>Python_3_10_11venv₩Scripts₩activate

(Python_3_10_11venv) 태그가 생성된다.

가상환경 종료하는 방법

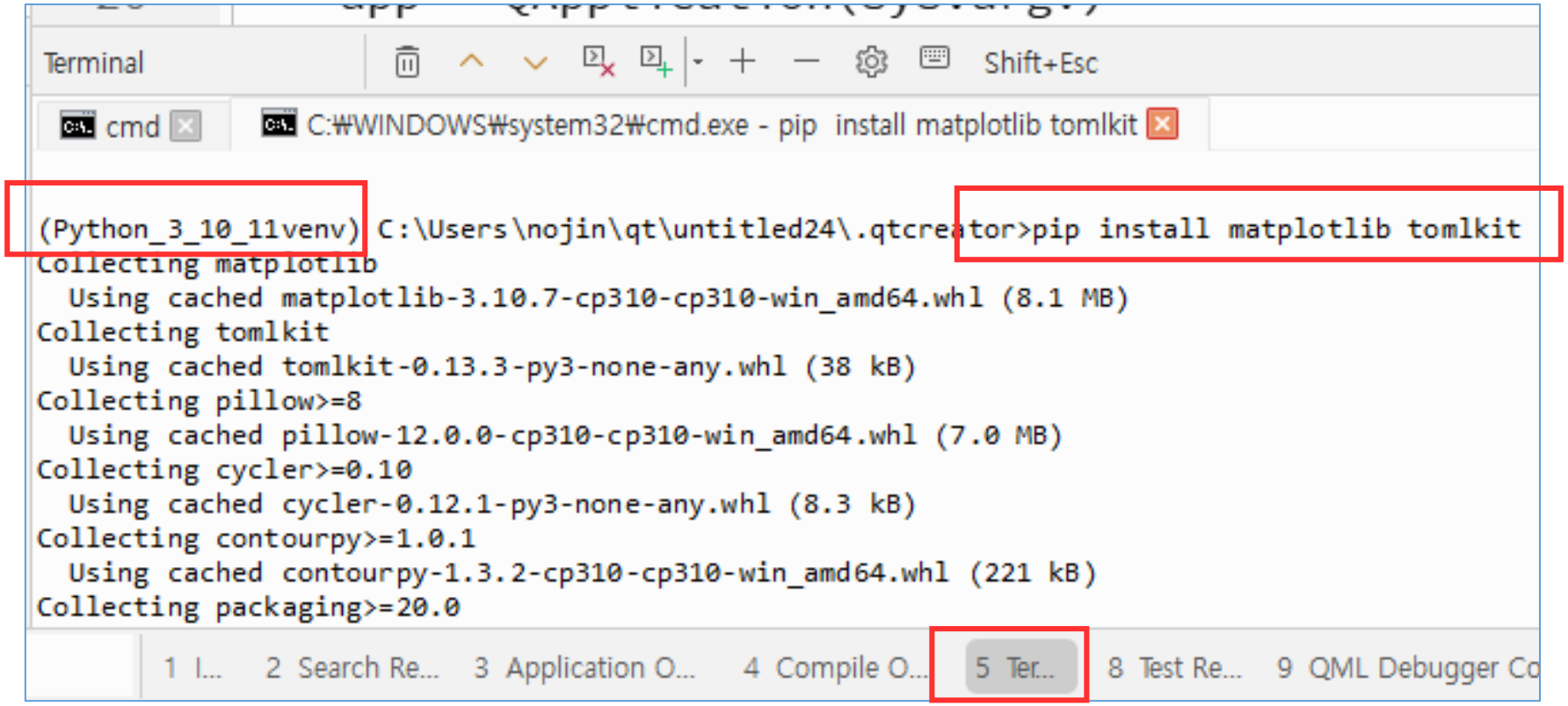
>deactivate



Confidential

Terminal 탭 클릭

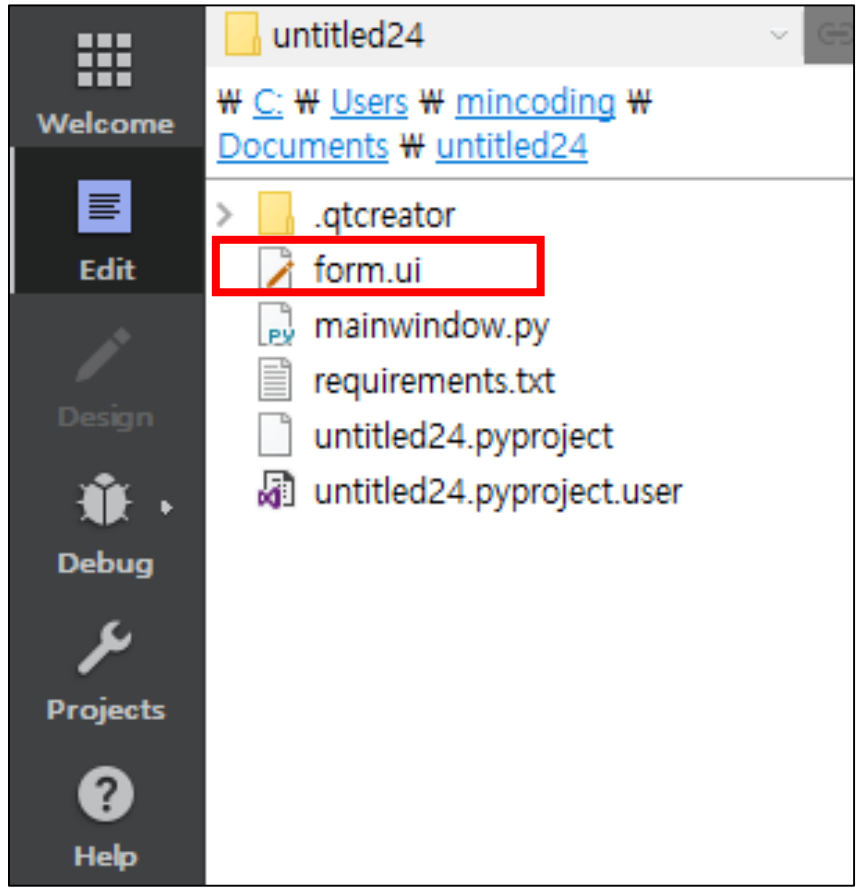
- pip install tomlkit
- 가상환경 확인



Confidential

project 경로 > form.ui 더블클릭(실행)

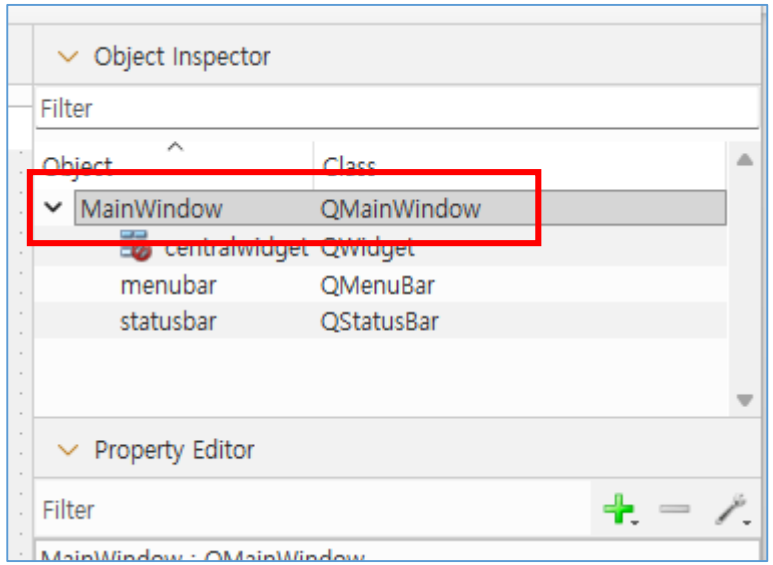
- **프로젝트 설정 시 같이 설치됨**

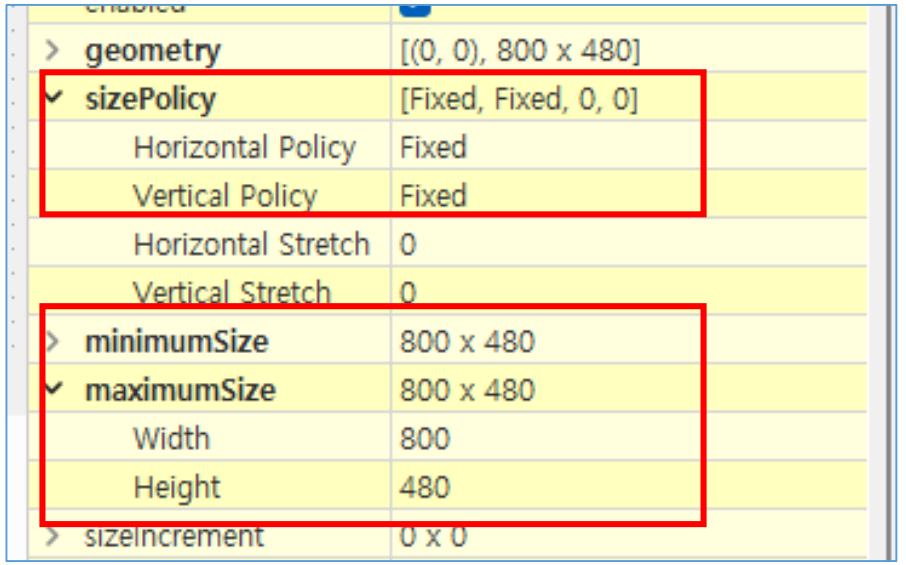


Confidential

MainWindow의 크기를 미리 지정한다.

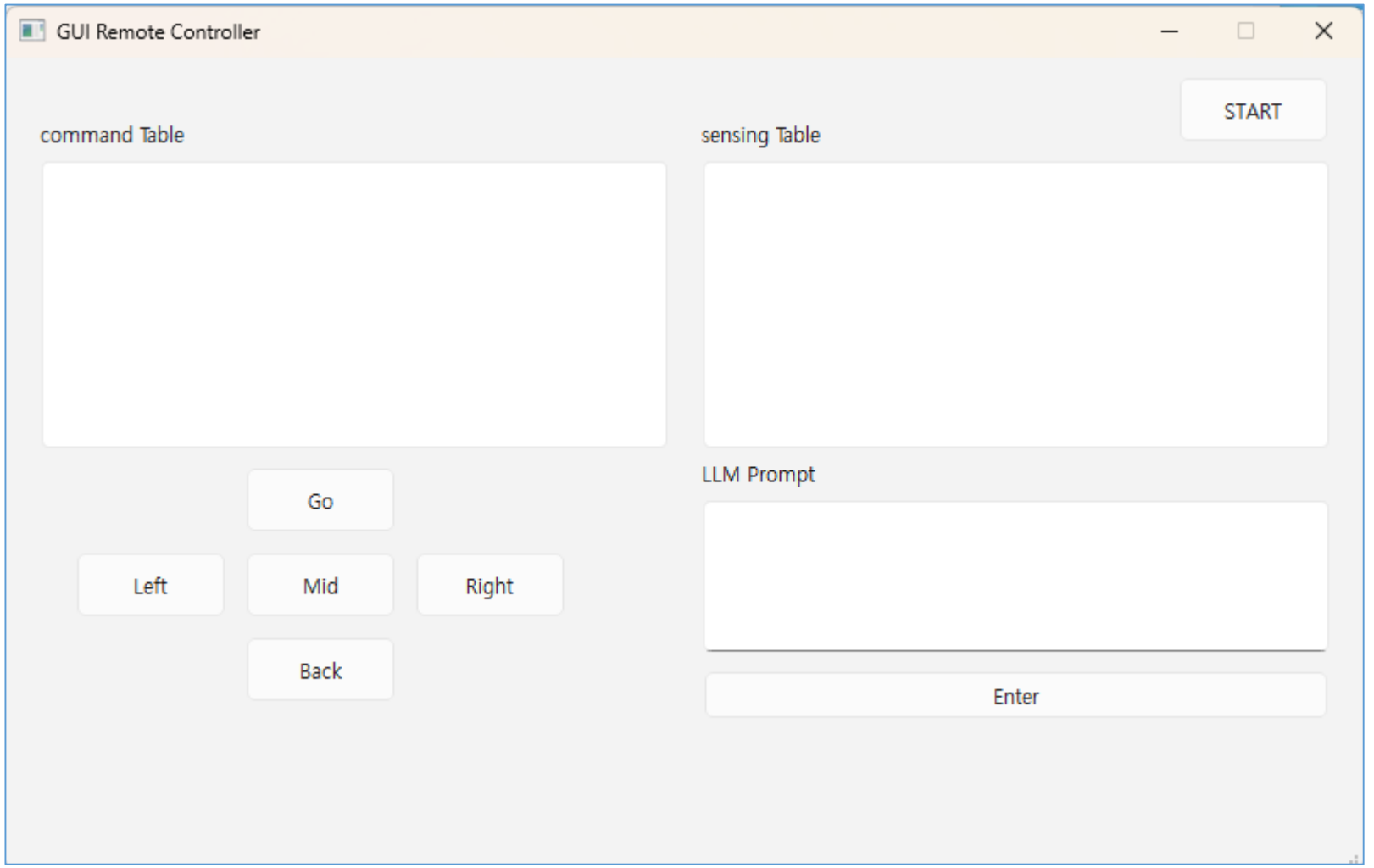
- **sizePolicy**
  - Horizontal Policy / Vertical Policy : Fixed
- **minimumSize / maximumSize**
  - Width : 800 / Height : 480





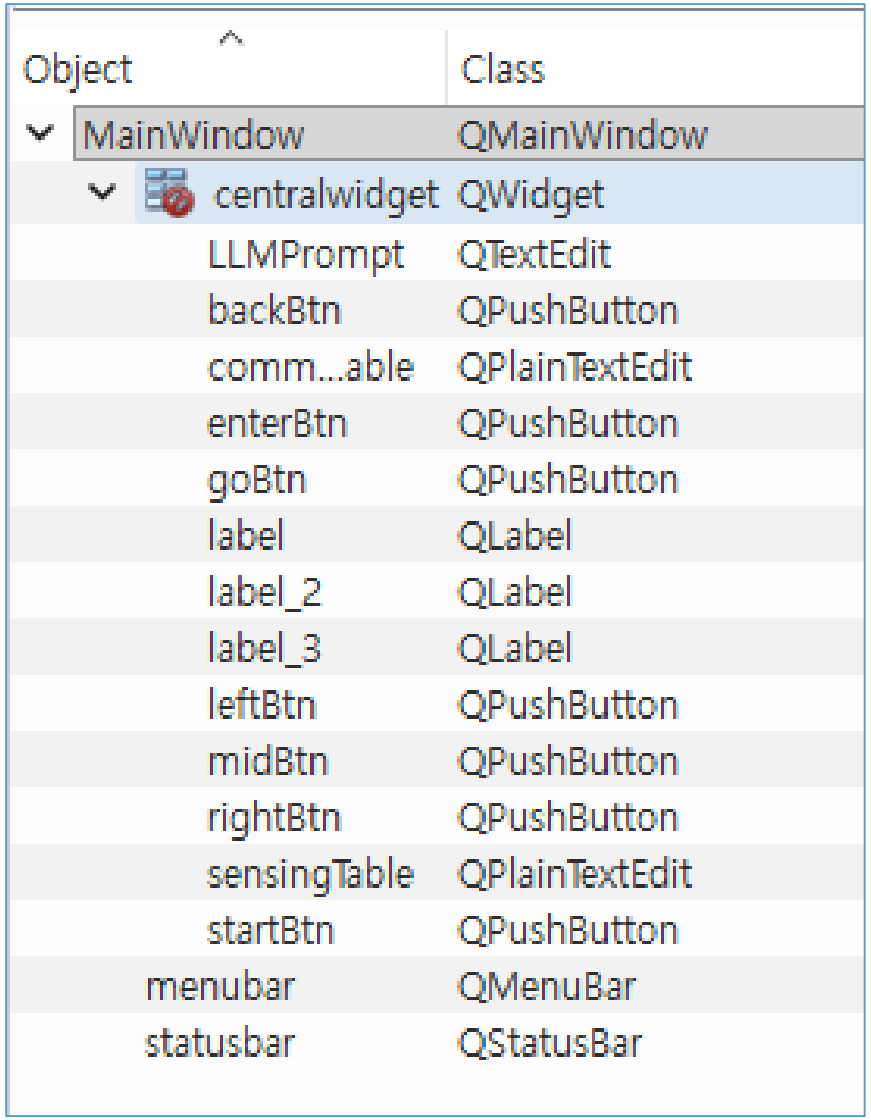
Confidential

QPushButton 7개, QPlainText 3개, QLabel 3개



Confidential

이름을 변경한다.



Object	Class
MainWindow	QMainWindow
centralwidget	QWidget
LLMPrompt	QTextEdit
backBtn	QPushButton
comm...able	QPlainTextEdit
enterBtn	QPushButton
goBtn	QPushButton
label	QLabel
label_2	QLabel
label_3	QLabel
leftBtn	QPushButton
midBtn	QPushButton
rightBtn	QPushButton
sensingTable	QPlainTextEdit
startBtn	QPushButton
menubar	QMenuBar
statusbar	QStatusBar

Confidential

시그널 / 슬롯을 등록한다. (7개)

Signals and Slots Editor

Sender	Signal	Receiver	Slot
backBtn	clicked()	MainWindow	back()
enterBtn	clicked()	MainWindow	enter()
goBtn	clicked()	MainWindow	go()
leftBtn	clicked()	MainWindow	left()
midBtn	clicked()	MainWindow	mid()
rightBtn	clicked()	MainWindow	right()
startBtn	clicked()	MainWindow	start()

Confidential

버튼을 눌러 간단한 테스트를 한다.

```
6   class MainWindow(QMainWindow):
7       def __init__(self, parent=None):
8           super().__init__(parent)
9           self.ui = Ui_MainWindow()
10          self.ui.setupUi(self)
11          self.init()
12
13      def init(self):
14          print('init')
15
16      def start(self):
17          print('start')
```

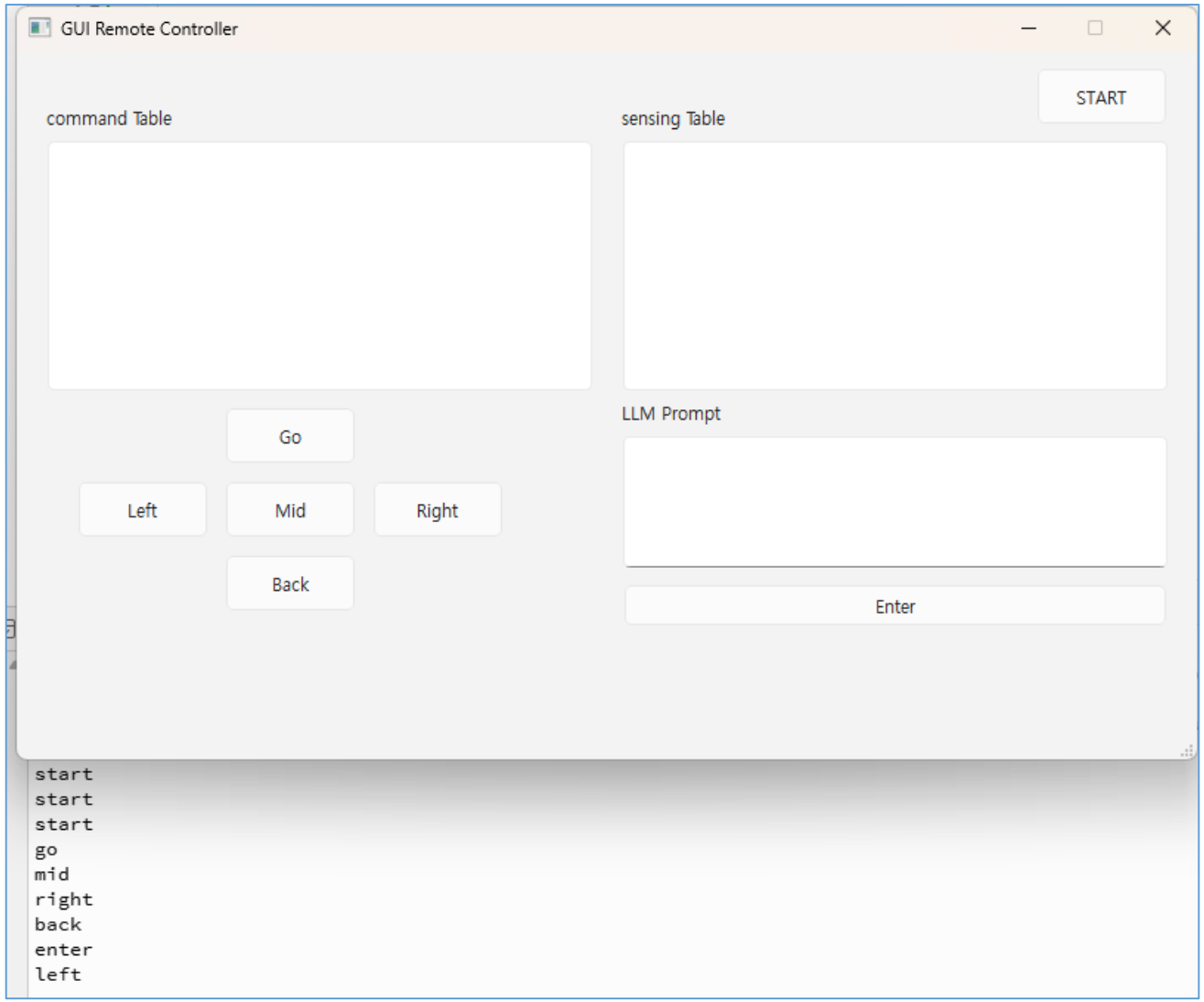
https://gist.github.com/hoconoco/e64d5fabe215824f408b2d2706b6b142

동작 결과

Confidential

버튼을 누르면

버튼의 텍스트가 출력된다.



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

commandData 생성하기

Confidential

User가 GUI 원격 제어기를 통해 Edge Device로 보내는 명령 데이터

- 현재 시간 정보
- 버튼 명령
- 명령 수행 여부 : 명령 수행 전 1, 수행 후 0
- +@로 value 전송

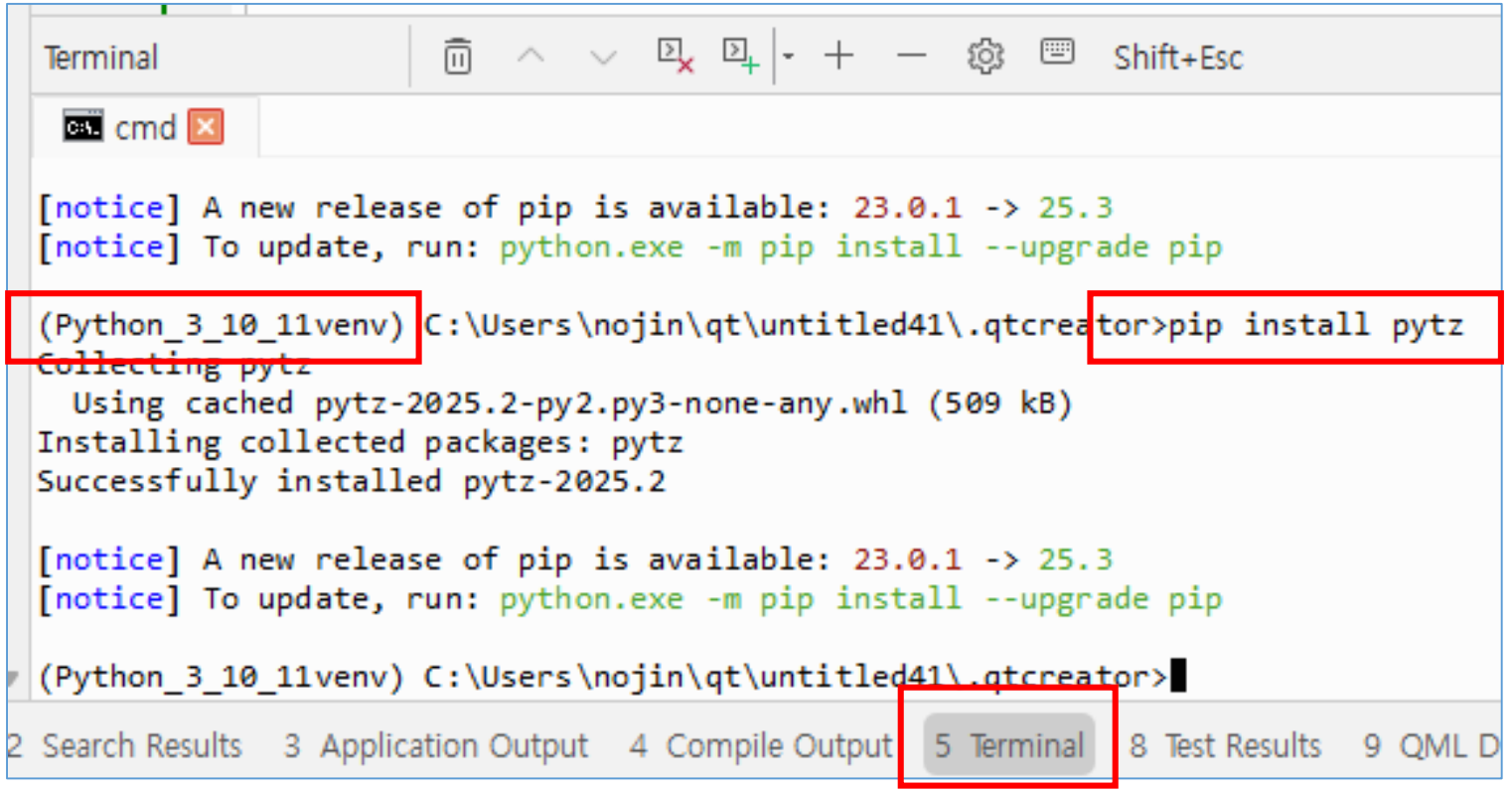
ex) Go 버튼 클릭 할 경우

```
commandData = {
 'time' : '2025-12-09 18:00:00',
'cmd_string' : 'go',
'is_finish' : 1,
'arg_string' : 5
}
```

```
commandData = {
    "time" : "None",
    "cmd_string" : "None",
    "is_finish" : 0,
    "arg_string" : 5
}
```

Terminal 탭을 클릭

- pip install pytz
- pytz : timezone 설정용 라이브러리



Confidential

현재 시간 정보와 함께 commandData를 생성한다.

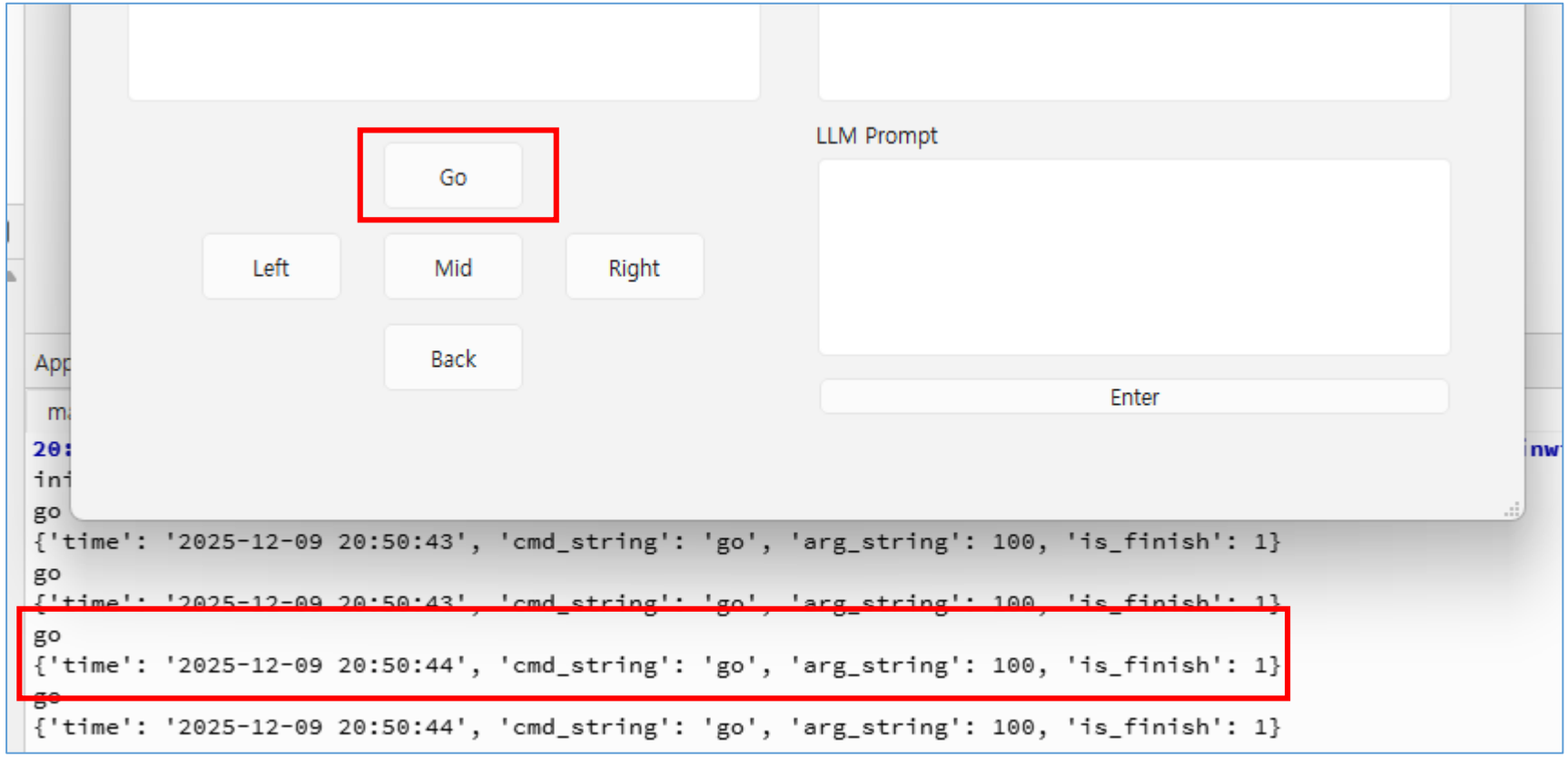
- **go button만 동작**

```
23        #commandData를 만드는 함수
24        #명령어, arg, 명령수행여부
25        def makeCommandData(self, str, arg, finish):
26            current_time = datetime.now(korea_timezone)
27            self.commandData["time"] = current_time.strftime("%Y-%m-%d %H:%M:%S")
28            self.commandData["cmd_string"] = str
29            self.commandData["arg_string"] = arg
30            self.commandData["is_finish"] = finish
31            return self.commandData
```

https://gist.github.com/hoconoco/e378375cf7bd1001db94aaabed0ce4bb

동작 결과

Go 버튼을 누르면, 현재 시간정보와 함께, commandData가 생성된다.



Confidential

라이브러리 호출

timezone 설정

```
6   from datetime import datetime
7   import pytz
8
9   # 한국 시간대 (Asia/Seoul)로 설정
10  korea_timezone = pytz.timezone("Asia/Seoul")
```

commandData용 dictionary 생성

```
12  class MainWindow(QMainWindow):
13
14      #commandData용 dictonary
15      commandData = dict()
```

Confidential

makeCommandData()

- commandData를 생성하는 함수
- 현재 시간 정보, 수행할 명령(str), 추가 전달 인자 (arg), 명령 수행여부 (finish)

```
23        #commandData를 만드는 함수
24        #명령어, arg, 명령수행여부
25        def makeCommandData(self, str, arg, finish):
26            current_time = datetime.now(korea_timezone)
27            self.commandData["time"] = current_time.strftime("%Y-%m-%d %H:%M:%S")
28            self.commandData["cmd_string"] = str
29            self.commandData["arg_string"] = arg
30            self.commandData["is_finish"] = finish
31            return self.commandData
```

Confidential

go()

- Go버튼 누르면, makeCommandData() 호출하여, commandData를 생성한다.
- 결과 출력

```
42          def go(self):
43              print('go')
44              self.commandData = self.makeCommandData("go", 100, 1)
45              print(self.commandData)
```

Confidential

다른 버튼도 똑같이 동작하게 한다.

- **start, enter 버튼 제외**

```
49      def mid(self):
50          self.commandData = self.makeCommandData("mid", 100, 1)
51          print(self.commandData)
52
53      def back(self):
54          self.commandData = self.makeCommandData("back", 100, 1)
55          print(self.commandData)
56
57      def left(self):
58          self.commandData = self.makeCommandData("left", 100, 1)
59          print(self.commandData)
```

https://gist.github.com/hoconoco/41747b19c4fd9d2aa5dfd37fe61d65ce

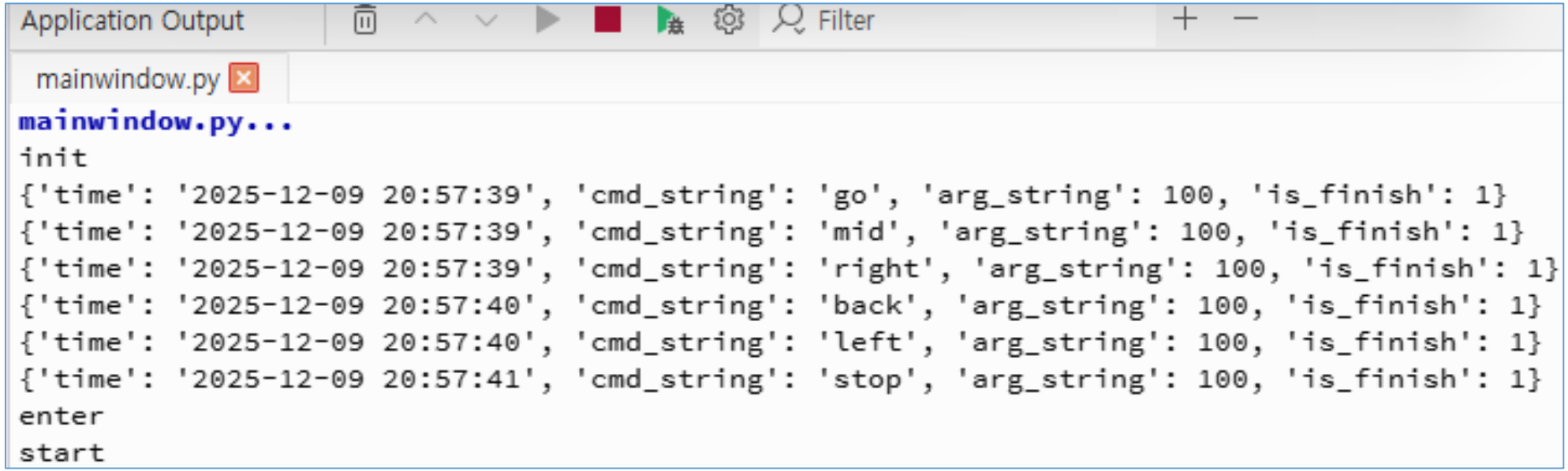
동작 결과

Confidential

start 버튼 : 프로그램의 실행

enter 버튼 : LLM에 프롬프트 넘길 때 사용

- 해당 버튼들은 기록하지 않아도 무방하기에 제외함.



SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

commandData 기록하기

Confidential

commandTable 에는
CommandData의 가장 최신 15개 정보만 표시할 예정이다.

GUI App은 비동기로 구현되어야 하며,
강의에서는 QThread를 사용해, UI를 업데이트할 예정이다.

Confidential

START 버튼을 누르면, Thread 시작

Go 버튼을 누르면, commandTable에 기록

- **다른 버튼은 아직 기록 되지 않는다.**

```
39        def settingUI(self):
40            #commandTable 초기화
41            self.ui.commandTable.clear()
42
43            #commandDataList 요소 하나씩 포매팅
44            for i in range(len(self.commandDataList)):
45                msg = "%3d | %s | %6s | %3d | %3d" % (
46                    i, self.commandDataList[i]["time"],
47                    self.commandDataList[i]["cmd_string"],
48                    self.commandDataList[i]["arg_string"],
49                    self.commandDataList[i]["is_finish"])
50                #commandTable에 표시
51                self.ui.commandTable.appendPlainText(msg)
```

https://gist.github.com/hoconoco/4c1981d10764f7e548bec42e195b5db2

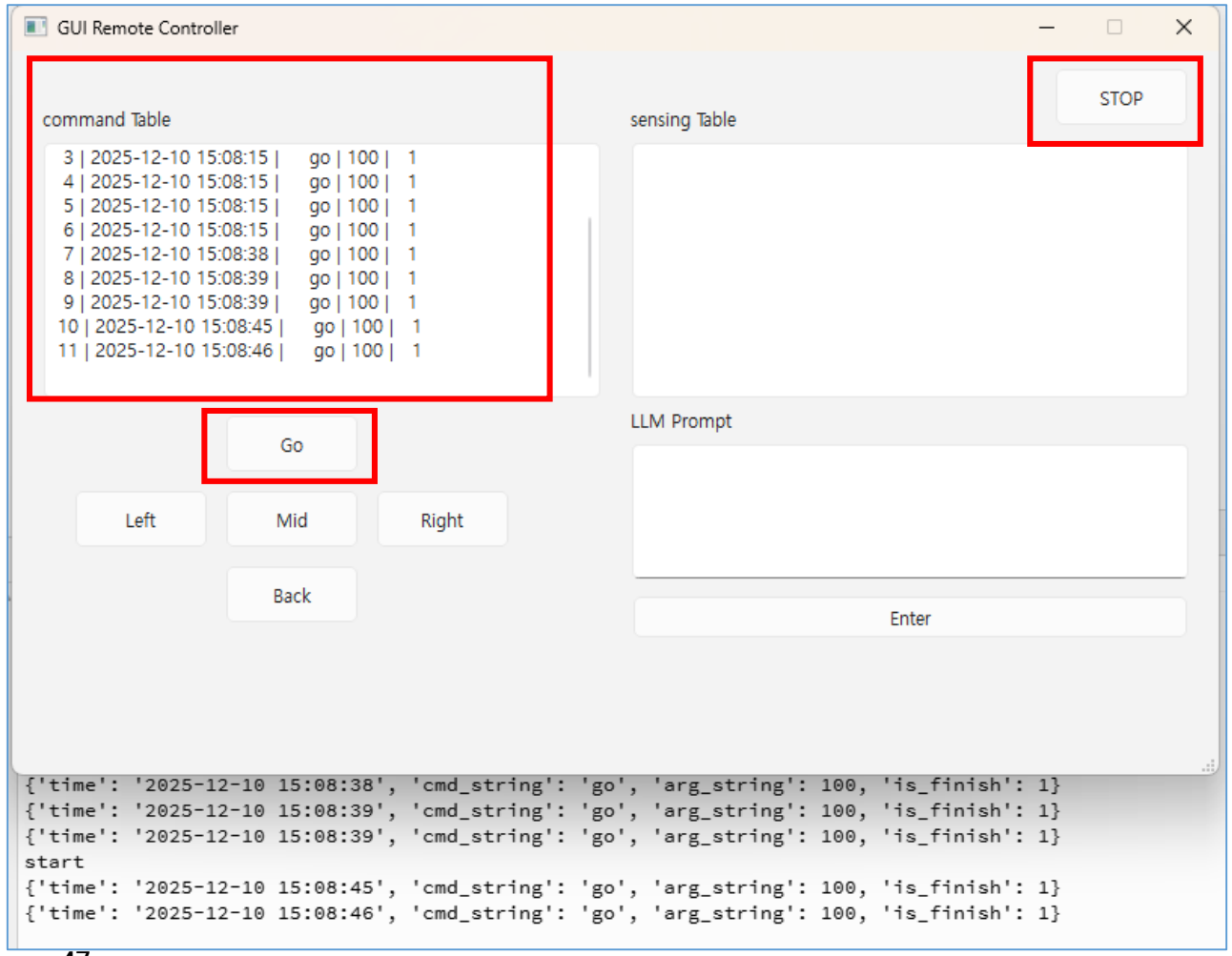
동작 확인

Confidential

1. START 버튼을 누른다.
   - STOP으로 변경
   - Thread 시작

2. Go 버튼을 누르면, commandTable에 기록된다.

3. STOP 버튼을 누른다.
   - Thread 정지
   - 기록 중지



Confidential

QThread/Signal 라이브러리 가져오기

```
 9   #QThread / Signal 가져오기
10   from PySide6.QtCore import QThread, Signal
11   #time 사용
12   import time
```

commandTable 출력용 list

```
39          #commandTable 출력용 리스트
40          commandDataList = list()
```

Confidential

UI Update용 Thread 생성

- 스레드 객체 생성되자마자 run
- 0.1초에 한번씩 signal 전송
  - UI update 요청
- stop 받으면 종료

```
16  #ui Update용 스레드
17  class UpdateThread(QThread):
18      # UI 갱신을 위한 Signal
19      update_signal = Signal()
20
21      def __init__(self):
22          super().__init__()
23          #객체 생성되자마자 동작
24          self.running = True
25
26      #start 받으면 바로 동작
27      def run(self):
28          while self.running:
29              self.update_signal.emit()    # UI 갱신 요청
30              time.sleep(0.1)              # 0.1초마다 반복
31
32      #stop 받으면 정지
33      def stop(self):
34          self.running = False
```

Confidential

settingUI()

- commandTable 초기화
- 출력 형식을 일정하게 하기 위한, Print formatting
- commandTable에 표시

```
39      def settingUI(self):
40          #commandTable 초기화
41          self.ui.commandTable.clear()
42
43          #commandDataList 요소 하나씩 포매팅
44          for i in range(len(self.commandDataList)):
45              msg = "%3d | %s | %6s | %3d | %3d" % (
46                  i, self.commandDataList[i]["time"],
47                  self.commandDataList[i]["cmd_string"],
48                  self.commandDataList[i]["arg_string"],
49                  self.commandDataList[i]["is_finish"])
50              #commandTable에 표시
51              self.ui.commandTable.appendPlainText(msg)
```

Confidential

start()

- **START 일 때,**
  - start 출력
  - STOP 변경
  - Thread 객체 생성
  - Thread 시작

- **STOP 일 때,**
  - stop 출력
  - START 변경
  - Thread 정지

UI Update를 on/off 한다.

```
74        def start(self):
75            if self.ui.startBtn.text() == "START":
76                print("START")
77                self.ui.startBtn.setText("STOP")
78                #Thread 객체 생성
79                self.update_thread = UpdateThread()
80                #Thread 의 signal과 settingUI() 연결
81                self.update_thread.update_signal.connect(self.settingUI)
82                #Thread 시작
83                self.update_thread.start()
84
85            elif self.ui.startBtn.text() == "STOP":
86                print("STOP")
87                self.ui.startBtn.setText("START")
88                #Thread 정지
89                self.update_thread.stop()
```

Confidential

go()

- commandDataList 에 commandData 추가
- 최신 15개 명령어만 남기기
- 기존 명령 초기화

```
65          def go(self):
66              self.commandData = self.makeCommandData("go", 100, 1)
67              print(self.commandData)
68
69              #commandDataList에 추가
70              self.commandDataList.append(self.commandData)
71              #최신 15개 명령어만 남기기
72              self.commandDataList = self.commandDataList[-15:]
73              #기존 명령 초기화
74              self.commandData = dict()
```

Confidential

closeEvent()

- **스레드 실행 중이라면, 정지 후 자원 정리**
- **hasattr(object, str)**
  - return bool
  - 객체가 특정 속성을 가지고 있는 지 체크

```
 98        def closeEvent(self, event):
 99            print("exit")
100            if hasattr(self, "update_thread"):
101                self.update_thread.stop()
102                self.update_thread.quit()
103                self.update_thread.wait()
104            event.accept()
```

Confidential

이제 모든 버튼의 명령이 commandTable에 기록된다.

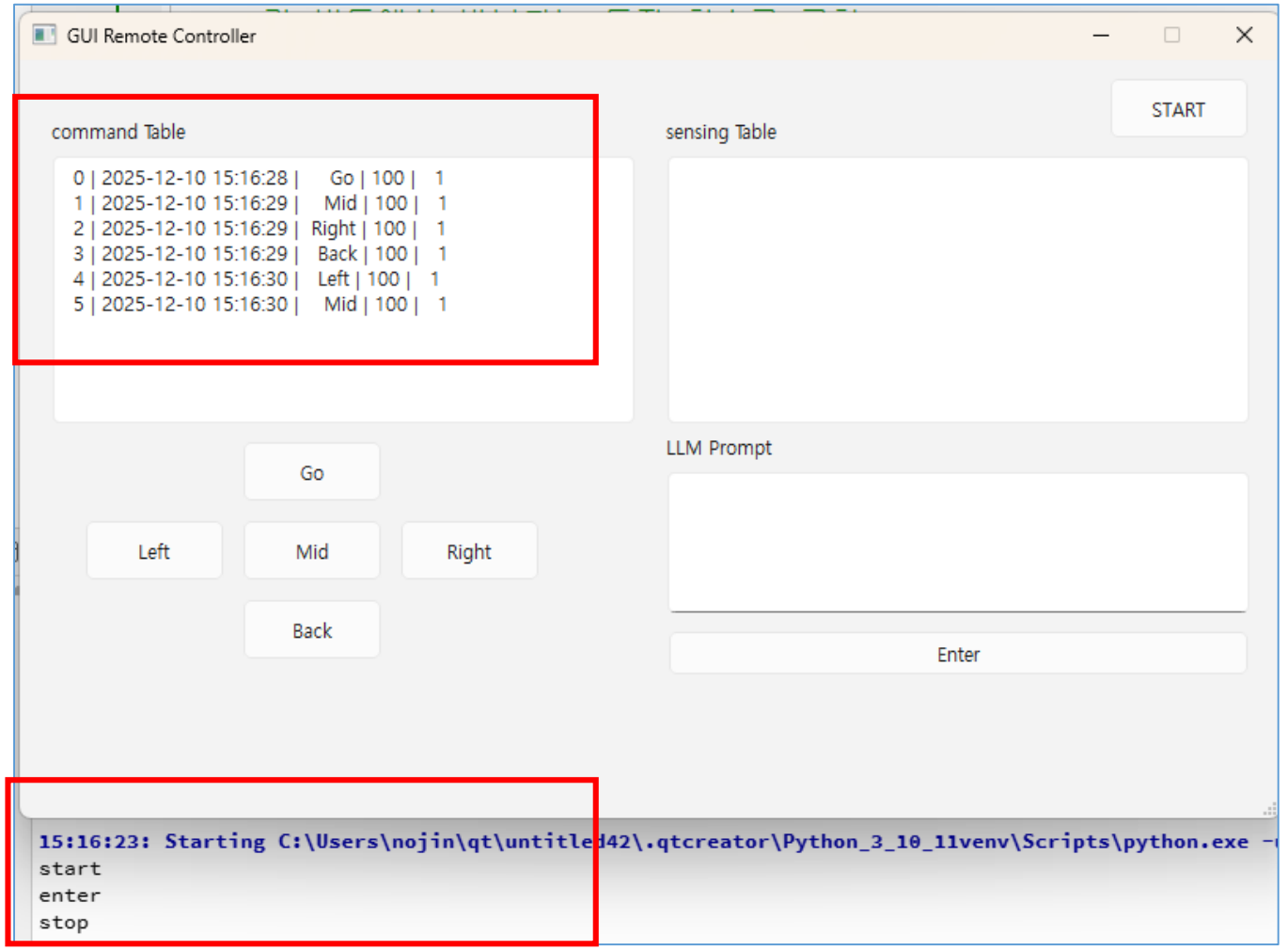
```
62        def back(self):
63            self.recordCommand(self.sender().text())
64
65        def left(self):
66            self.recordCommand(self.sender().text())
67
68        def right(self):
69            self.recordCommand(self.sender().text())
```

https://gist.github.com/hoconoco/e82dac1227f5117d666b4e24fd1666c0

동작 결과

이제 User의 명령이 기록되기 시작한다.

- 기록 되지 않는 버튼
- START
- Enter



코드 설명

Confidential

.sender()

- 시그널 발생시킨 위젯 알 수 있음

.text()

- 해당 버튼에 적힌 text()

recordCommand()

- 각 버튼마다 반복되는 동작을 함수로 처리
- 버튼에 적힌 텍스트 값을 읽어온다.

```
68          def right(self):
69              self.recordCommand(self.sender().text())
```

```
79          #반복되는 동작을 함수로 처리
80          def recordCommand(self, name):
81              self.commandData = self.makeCommandData(name, 100, 1)
82              self.commandDataList.append(self.commandData)
83              self.commandDataList = self.commandDataList[-15:]
84              self.commandData = dict()
```

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로! Enabling People

GUI 원격 제어기 with DB

챕터의 포인트

- FireStore Database
- DB 연결하기

SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

FireStore Database

Confidential

목표

- Firebase의 최신 Database인 FireStore에 데이터를 업로드한다.
- 인증 방법을 등록하여, DB를 관리한다.

Firebase에는 두 개의 Database가 있다.

- **Realtime Database (Legacy) : 인증 X**
- **Firestore Database (최신) : 인증 필요 → 보안 강화**

우리는 이번 시간에 Firestore Database를 사용할 예정이다.

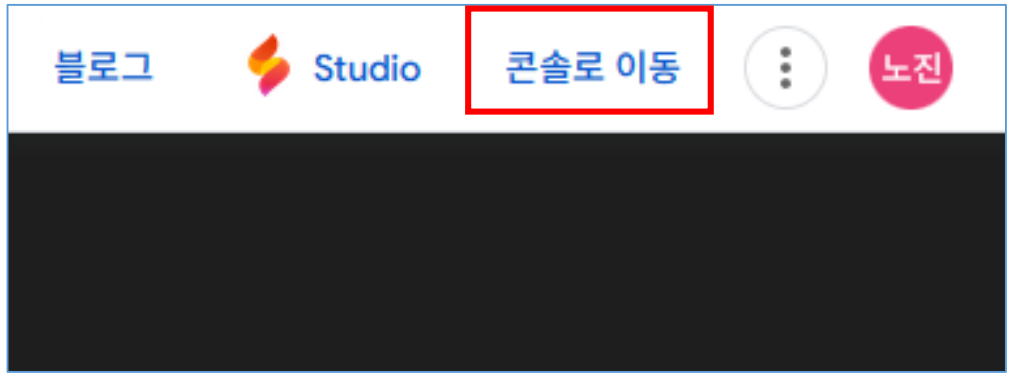
Firebase에 로그인한다.

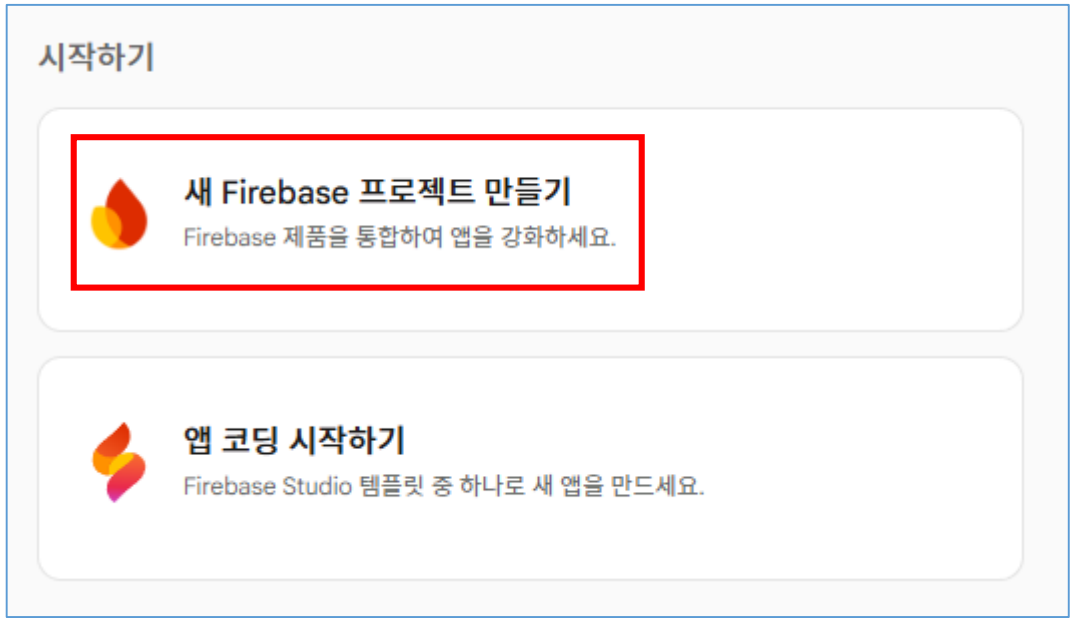
- https://firebase.google.com/



Confidential

콘솔로 이동 클릭

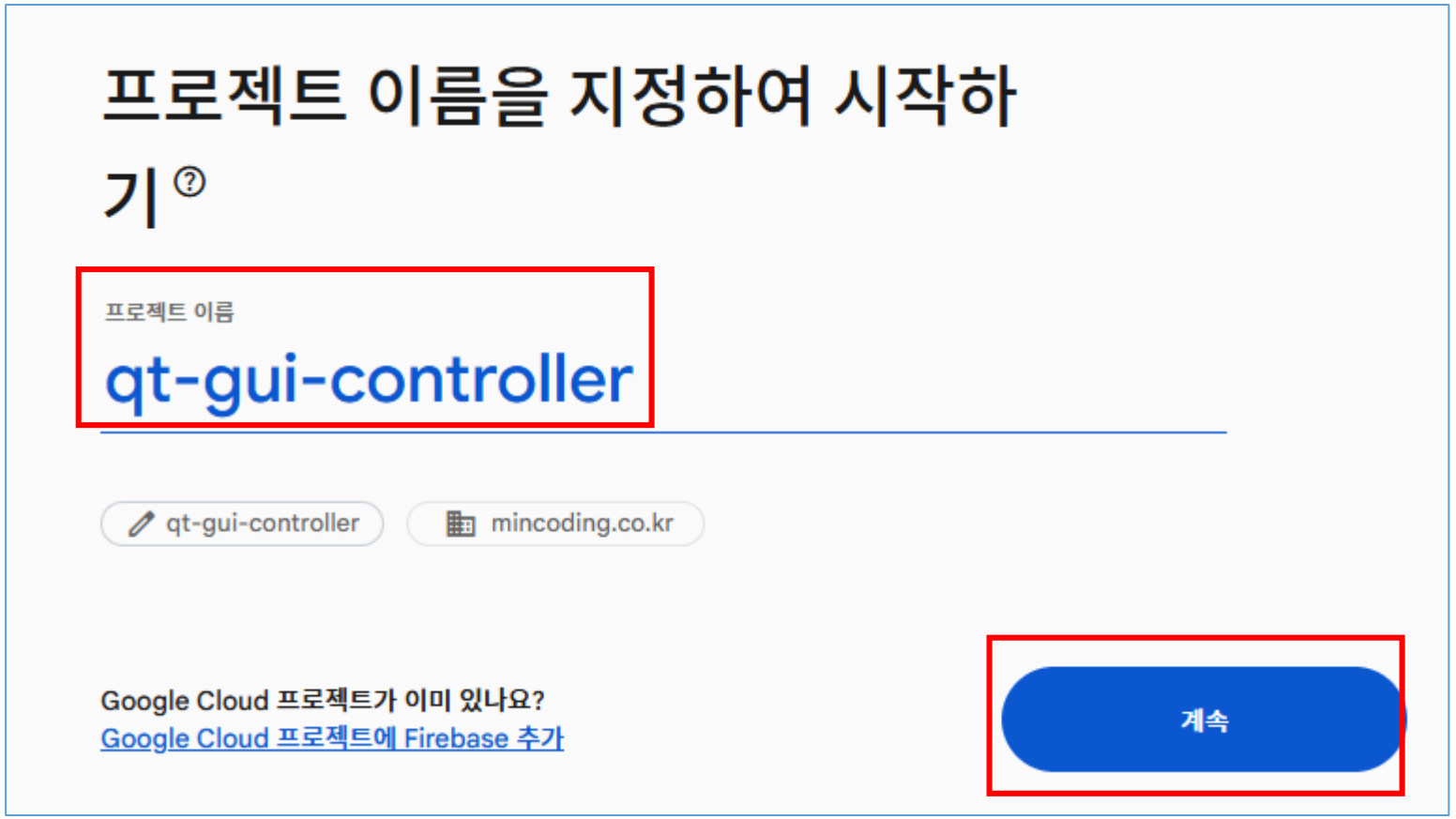




Confidential

프로젝트 이름 작성

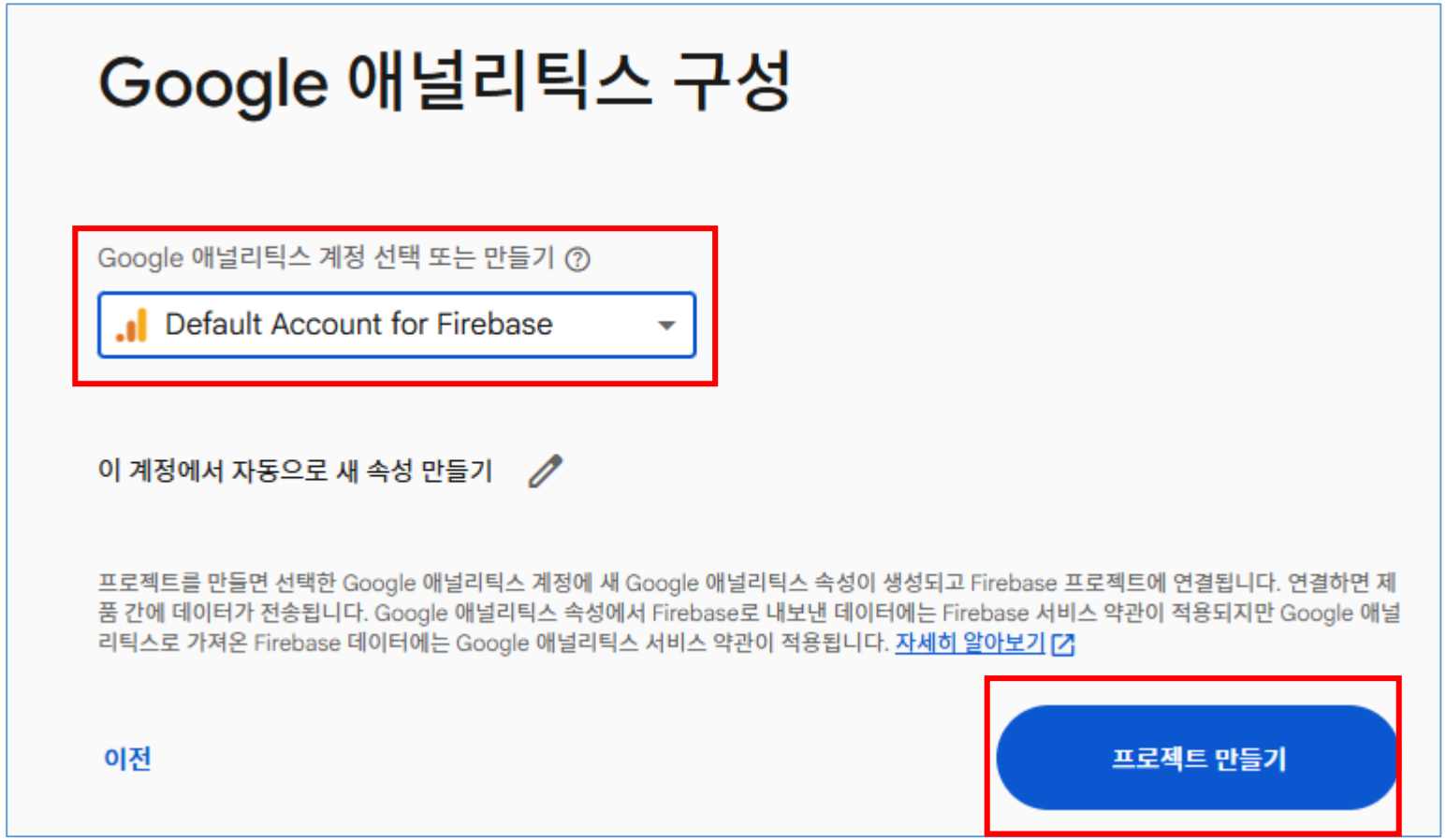
계속 클릭



Confidential

가장 중요한 시간이다.

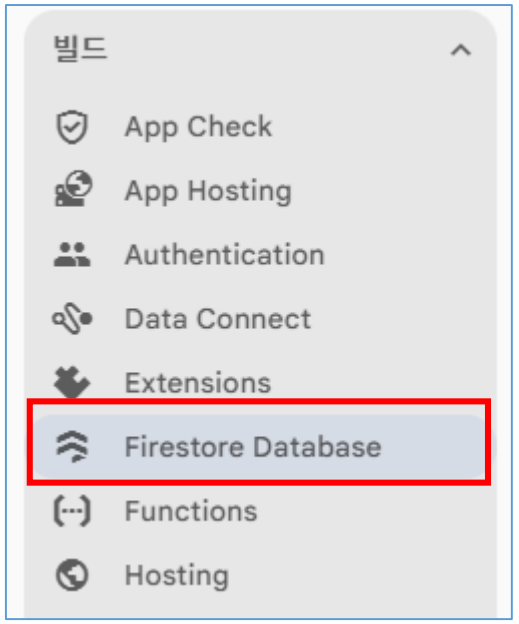
- **Firebase는 serverless로 API 호출마다 요금을 청구하는 방식이다.**



빌드 탭 클릭

- Firestore Database 클릭

데이터베이스 만들기 클릭





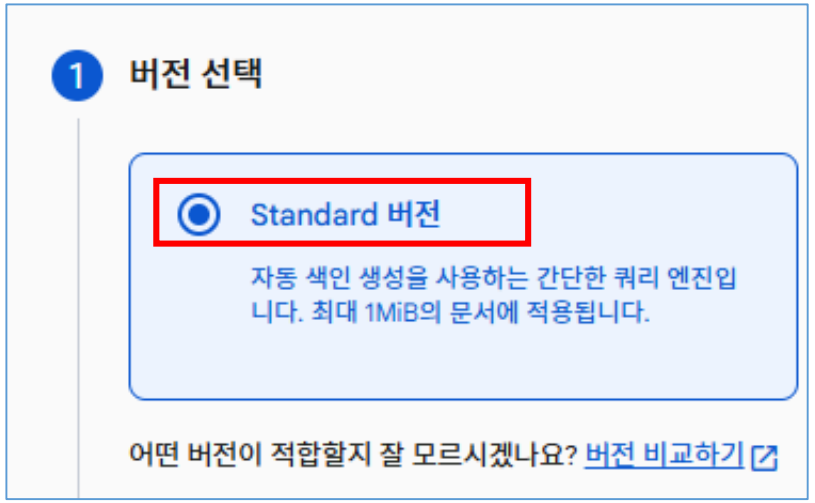
데이터베이스 만들기

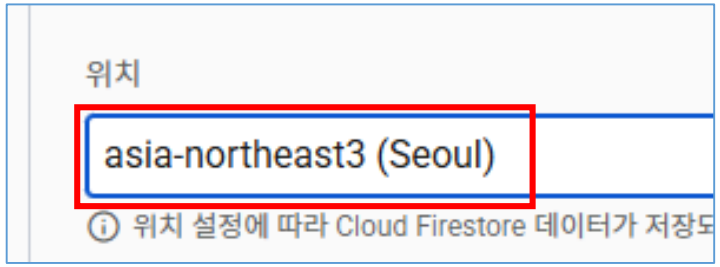
Confidential

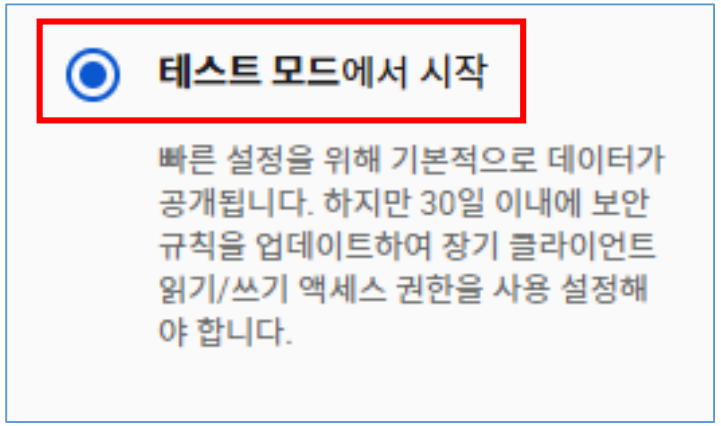
버전 : Standard 버전

위치 : asia-northeast3 (Seoul)

테스트모드 시작



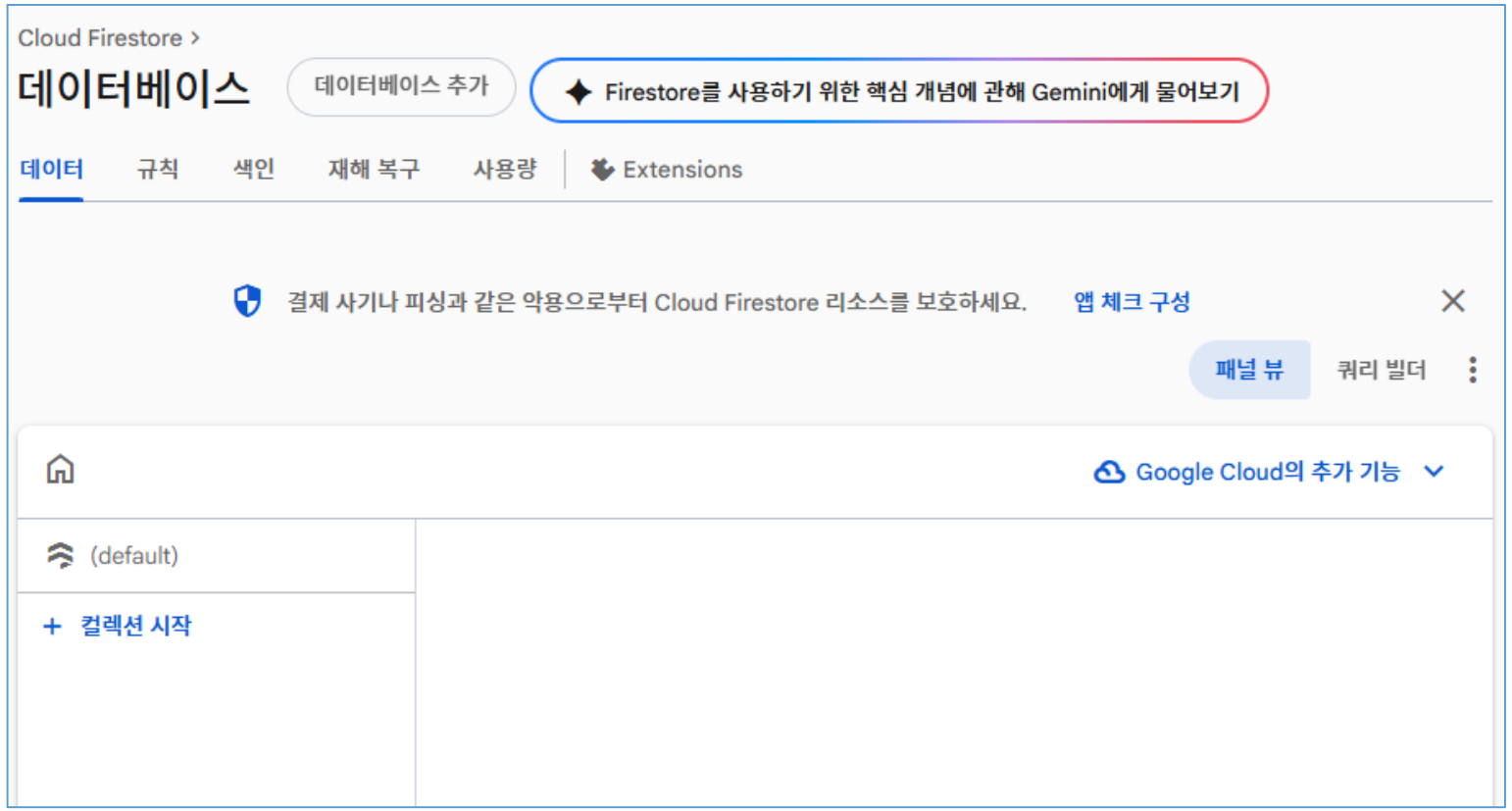




Confidential

FireStore의 데이터 접근 방식

- 컬렉션 → 문서 → Data
- 직접 data 업로드해서 살펴본다.



Firebase Admin SDK 다운로드

Confidential

Firebase의 Database는 패스워드로 접근하는 방식이 아닌,

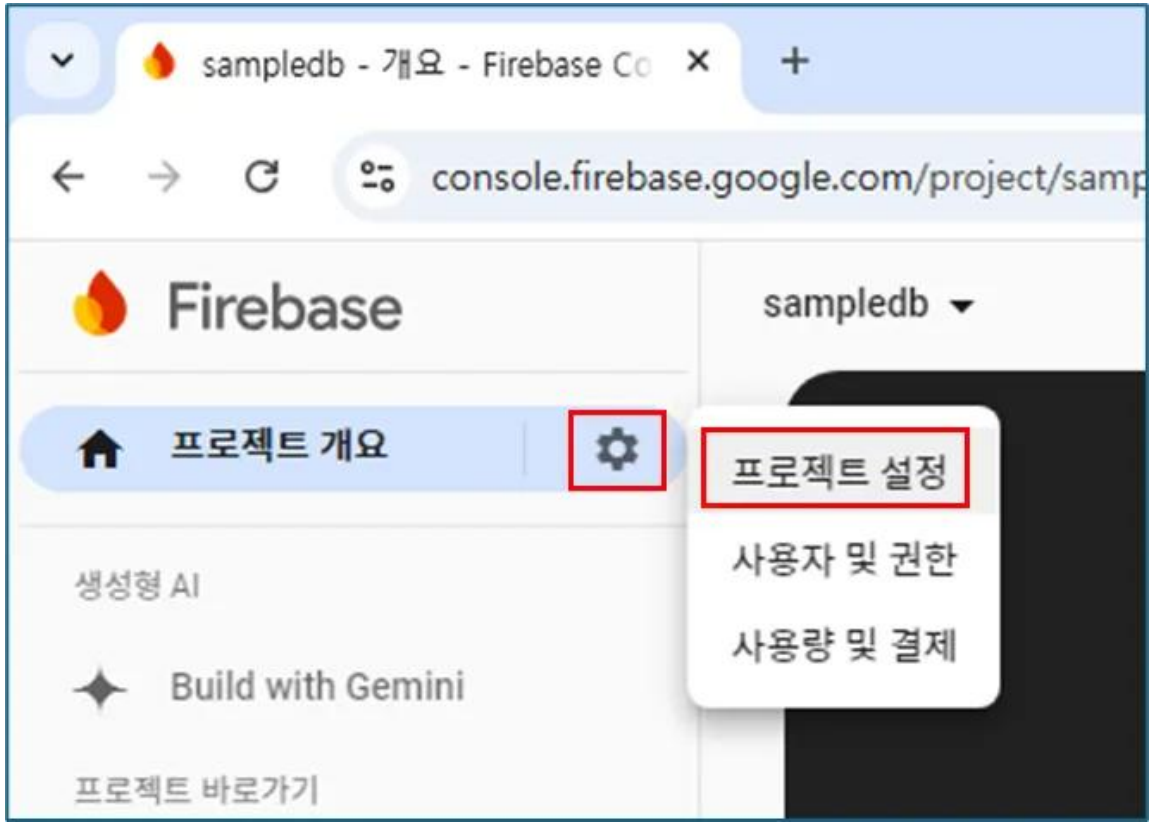
인증 토큰 or 서비스 계정 기반으로 권한을 관리한다.

- **서버에서 Firestore에 접근할 때 Firebase는 Admin SDK 를 사용하며, SDK는 보안 규칙을 우회할 수 있는 서버 권한을 제공한다.**

Confidential

화면 상단 톱니바퀴 클릭

프로젝트 설정 클릭



Firebase Admin SDK 다운로드 2

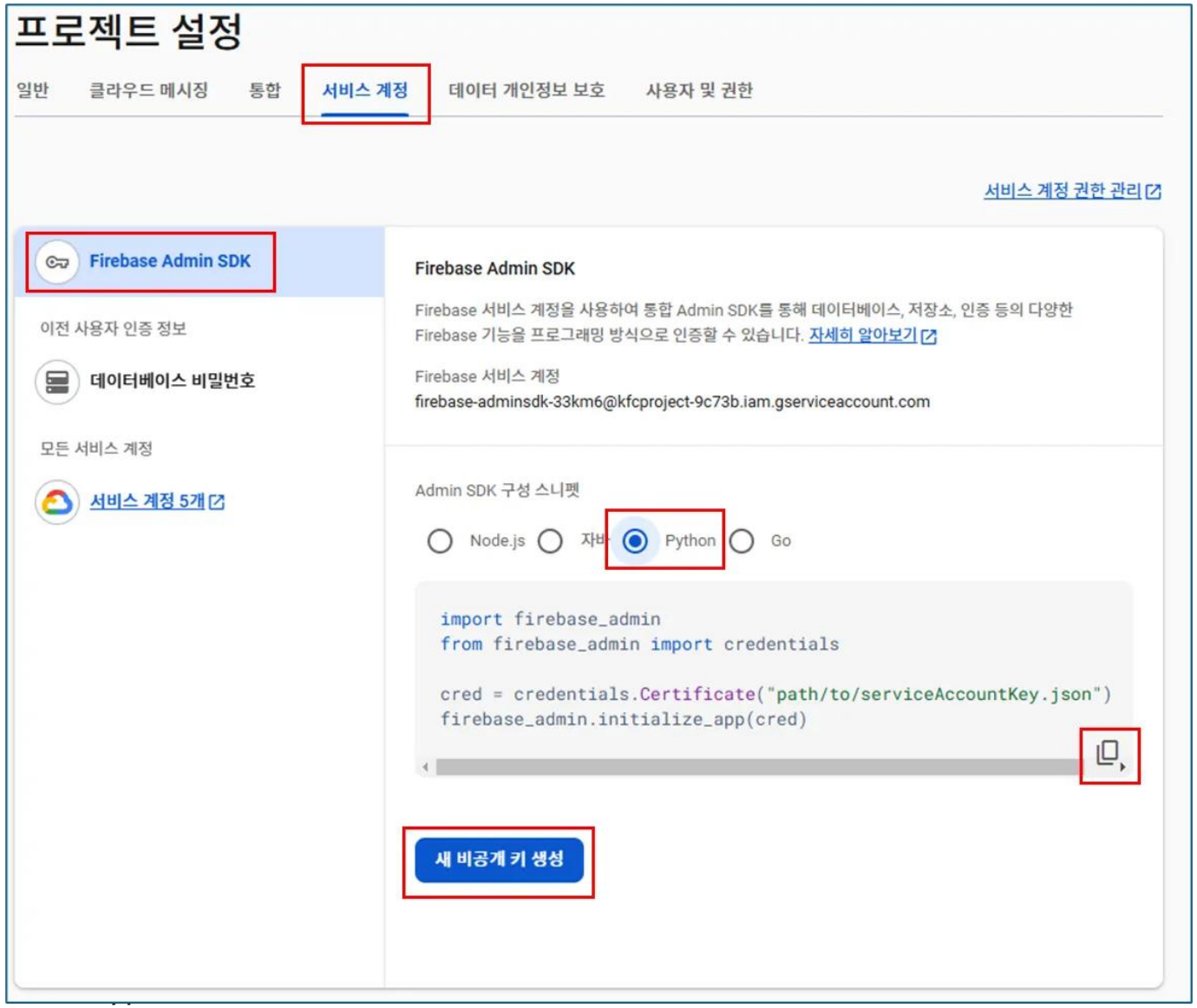
Confidential

서비스 계정 탭 클릭

Python 체크

- 샘플 코드 복사

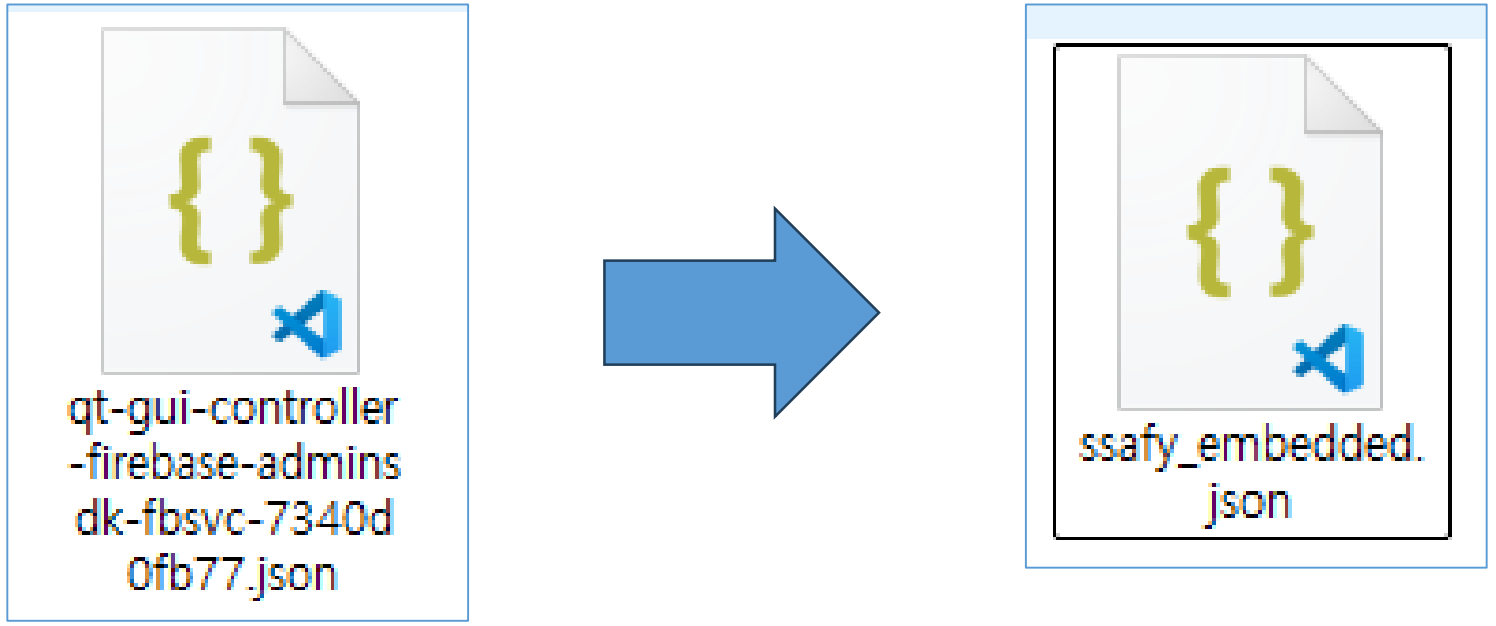
새 비공개 키 생성 클릭



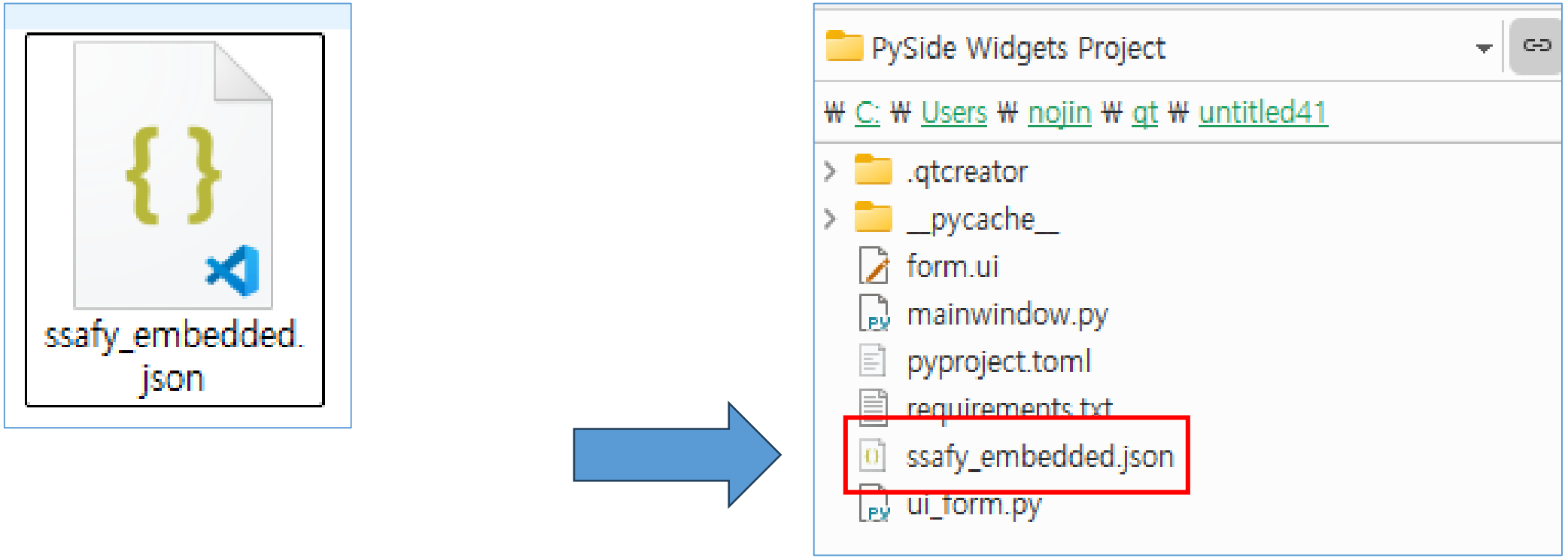
비공개 키 이름 변경

비공개 키(.json) 이름을 쉽게 변경한다.

- **절대 외부 공개를 해선 안된다.**



Qt 프로젝트에 .json 파일을 복사한다.



SAMSUNG SW AI ACADEMY FOR YOUTH

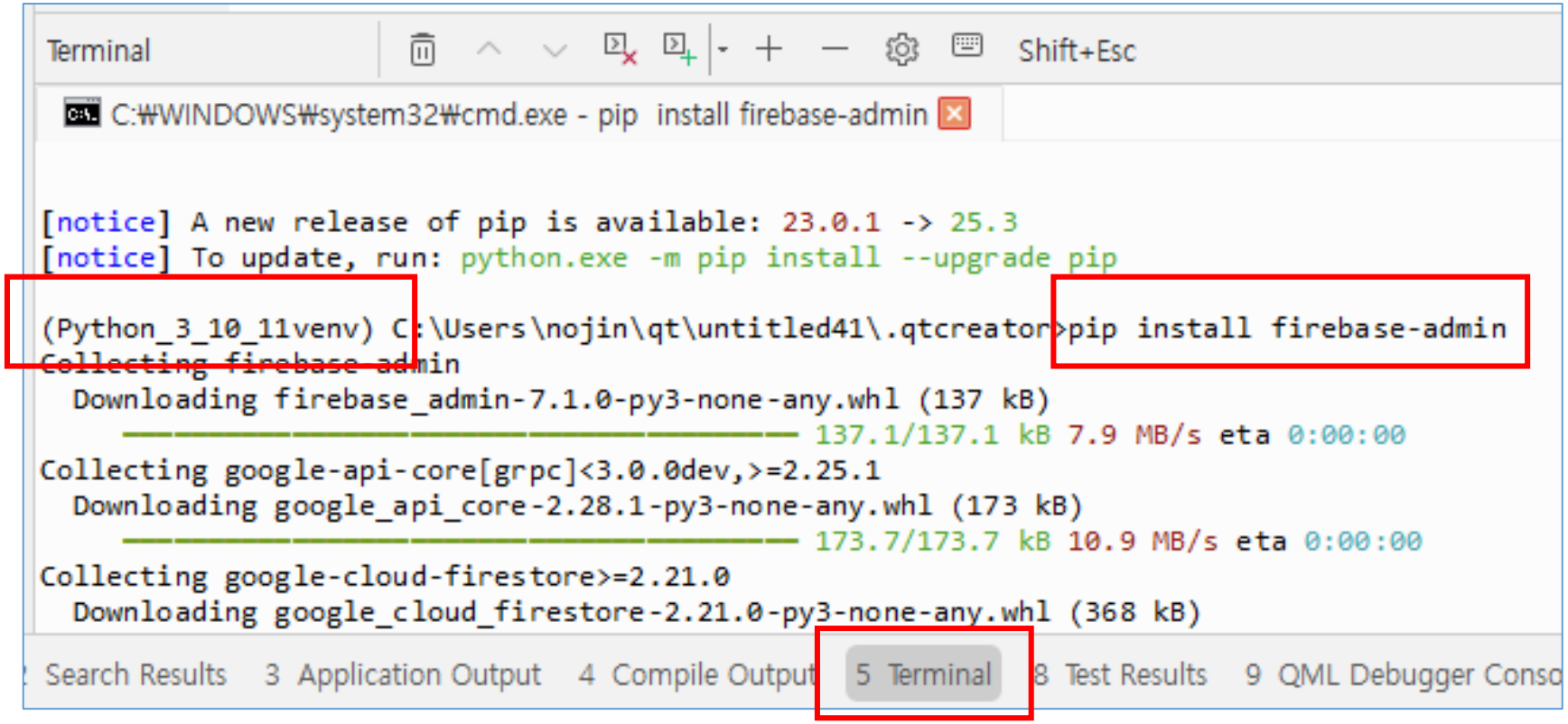
함께가요 미래로!
Enabling People

DB 연결하기

Confidential

Terminal 탭 클릭

- pip install firebase-admin
- 가상환경 체크



Confidential

FireStore에 샘플 데이터 업로드

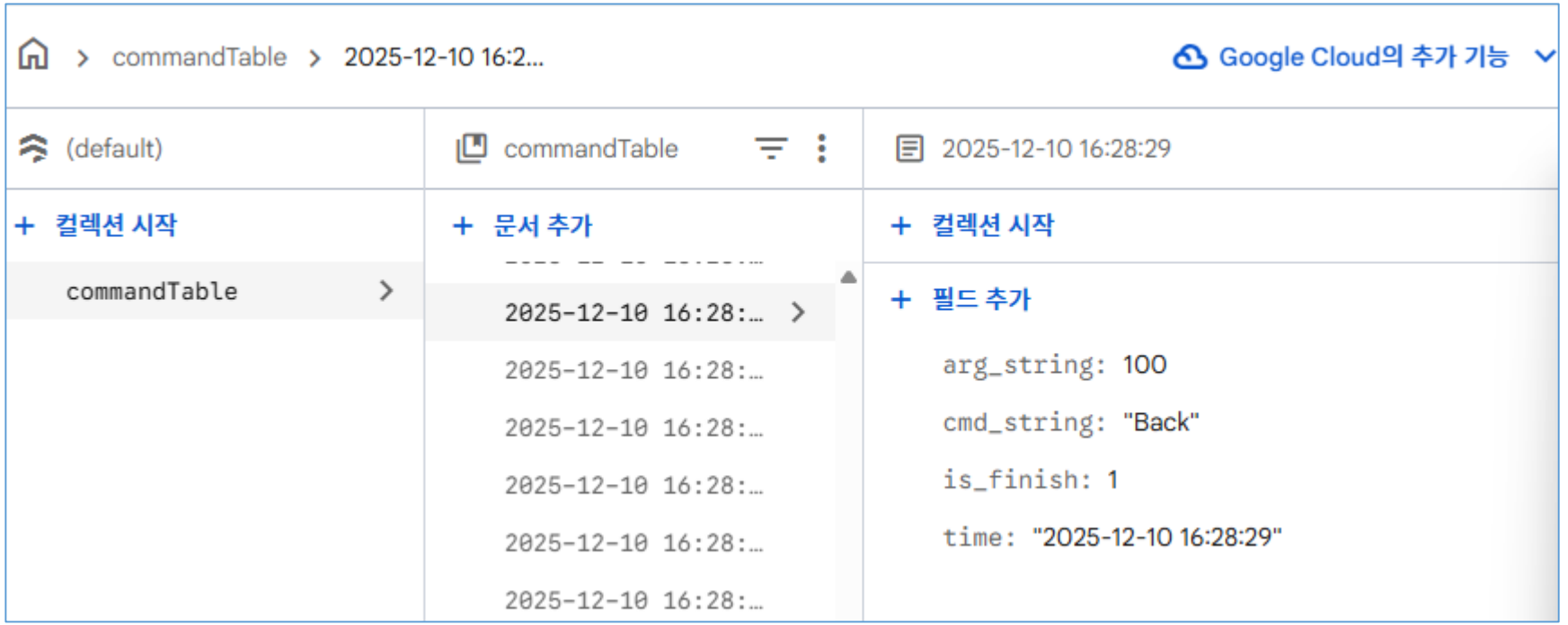
```
38  class UploadData(QThread):
39      # 업로드 완료 시 메인 스레드로 문서 ID 전달하는 Signal
40      upload_complete = Signal(str)
41
42      def __init__(self):
43          super().__init__()
44          self.running = True
45          # 업로드 대기 데이터 저장용 리스트
46          self.queue = []
47
48      def run(self):
49          while self.running:
50              # 업로드 요청이 있으면
51              if self.queue:
52                  # 큐에서 맨 앞 요청 꺼내기 (FIFO)
53                  item = self.queue.pop(0)
54                  # Firestore 업로드 처리
55                  self.upload_to_firestore(item)
56              # 과도한 CPU 사용 방지용 딜레이
57              time.sleep(0.05)
```

https://gist.github.com/hoconoco/7451b52c3c8d270baec5c0b230506b6b

Confidential

DB에 데이터가 업로드 되기 시작했다.

- FireStore는 컬렉션 - 문서 - Data 순으로 정리된다.
- 컬렉션은 data 종류
- 문서에 시간 정보를 두어 구분



Confidential

Firebase_admin 라이브러리 추가

서비스계정 키 등록

- 인증

Admin SDK 초기화

- SDK 준비

클라이언트 초기화

- 서버와 통신하기 위한 client 객체 생성

```
 9   import firebase_admin
10   from firebase_admin import credentials
11   from firebase_admin import firestore
12
13   # 서비스 계정 키 파일 경로를 설정
14   # key 이름은 각 팀마다 생성한 키 이름으로 변경한다.
15   path = '.json'
16   cred = credentials.Certificate(path)
17   # Firebase Admin SDK 초기화
18   firebase_admin.initialize_app(cred)
19   # Firestore 클라이언트 초기화
20   db = firestore.client()
```

Confidential

UploadData() 클래스

- Firebase 로 데이터를 업로드하는 클래스
- 데이터 업로드는 비동기로 진행되어야 한다.

- 메인 Thread로 전송 완료를 보낼 시그널 생성
- 여러 데이터를 순차적으로 업로드 할 queue 생성

- run()
  - 업로드 요청이 있으면, 앞에서부터 처리

```
39  class UploadData(QThread):
40      # 업로드 완료 시 메인 스레드로 문서 ID 전달하는 Signal
41      upload_complete = Signal(str)
42
43      def __init__(self):
44          super().__init__()
45          self.running = True
46          # 업로드 대기 데이터 저장용 리스트
47          self.queue = []
48
49      def run(self):
50          while self.running:
51              # 업로드 요청이 있으면
52              if self.queue:
53                  # 큐에서 맨 앞 요청 꺼내기 (FIFO)
54                  item = self.queue.pop(0)
55                  # Firestore 업로드 처리
56                  self.upload_to_firestore(item)
57              # 과도한 CPU 사용 방지용 딜레이
58              time.sleep(0.05)
```

Confidential

UploadData() 클래스

- **request_upload()**
  - 버튼을 눌러서 데이터 업로드 요청 받으면 실행
  - 큐에 업로드 요청 추가
- **upload_to_firestore()**
  - 업로드 정보 파싱
    - 컬렉션, 문서, 데이터
  - 문서 접근
  - 데이터 저장
  - 데이터 업로드 후 시그널 발송
- **try~except : 예외 처리 구문**

```
60      # 명령 버튼 누르면 호출
61      def request_upload(self, collection, document_id, data):
62          # 업로드 요청을 큐에 추가
63          self.queue.append((collection, document_id, data))
64
65      def upload_to_firestore(self, item):
66          # 업로드 정보 파싱
67          collection, document_id, data = item
68          try:
69              # Firestore 문서 참조
70              doc_ref = db.collection(collection).document(document_id)
71              # Firestore 문서에 데이터 저장
72              doc_ref.set(data)
73              print(f"[Firestore] {collection}/{document_id} 저장됨")
74              # 업로드 완료 Signal 발생
75              self.upload_complete.emit(document_id)
76          except Exception as e:
77              print("Firestore 업로드 오류:", e)
78
79      def stop(self):
80          self.running = False
```

Confidential

recordCommand()

- 버튼을 눌러 기록할 때, Firebase의 업로드 요청도 같이 한다.
- commandTable - 시간 - commandData

```
116         def recordCommand(self, name):
117             self.commandData = self.makeCommandData(name, 100, 1)
118             self.commandDataList.append(self.commandData)
119             self.commandDataList = self.commandDataList[-15:]
120 
121             # Firestore 업로드 요청
122             doc_id = self.commandData["time"]
123             self.fs_thread.request_upload("commandTable", doc_id, self.commandData.copy())
124 
125             self.commandData = dict()
```

코드 설명

Confidential

start()

- START일 때,
  - 업로드 스레드 객체 생성
  - 스레드 시작
  - 스레드에서 받을 시그널의 슬롯 등록
- STOP일 때,
  - 스레드 정지

데이터 기록과 업로드는 같이 진행된다.

```
127         def start(self):
128             if self.ui.startBtn.text() == "START":
129                 print("START")
130                 self.ui.startBtn.setText("STOP")
131                 self.update_thread = UpdateThread()
132                 self.update_thread.update_signal.connect(self.settingUI)
133                 self.update_thread.start()
134
135                 # Firestore Upload Thread
136                 self.fs_thread = UploadData()
137                 # 업로드 스레드 시작
138                 self.fs_thread.start()
139                 self.fs_thread.upload_complete.connect(self.upload)
140
141             elif self.ui.startBtn.text() == "STOP":
142                 print("STOP")
143                 self.ui.startBtn.setText("START")
144                 self.update_thread.stop()
145                 #업로드도 정지
146                 self.fs_thread.stop()
```

코드 설명

Confidential

closeEvent()

- 창 닫을 때, 업로드 스레드 종료 후 자원 정리

upload()

- 업로드 완료 시 시그널을 받으면, 출력된다.

```
170         def closeEvent(self, event):
171             print("exit")
172             if hasattr(self, "update_thread"):
173                 self.update_thread.stop()
174                 self.update_thread.quit()
175                 self.update_thread.wait()
176 
177             # Firestore thread 종료
178             if hasattr(self, "fs_thread"):
179                 self.fs_thread.stop()
180                 self.fs_thread.quit()
181                 self.fs_thread.wait()
```

```
148         # Firestore 업로드 완료 시 메시지
149         def upload(self, doc_id):
150             print(f"[업로드 완료] 문서 ID: {doc_id}")
```

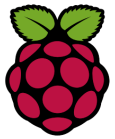
SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

라즈베리파이 포팅

폴더를 만들고, 시간 정보를 동기화 한다.

Confidential



라즈베리파이5를 연결하고 폴더를 만든다.

- **mkdir controller**
- **cd ./controller**

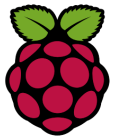
```
pi@raspberrypi:~ $ mkdir controller
pi@raspberrypi:~ $ cd ./controller/
pi@raspberrypi:~/controller $
```

시간 동기화

- **멀티캠퍼스 사업장 (수동 동기화)**
  - sudo ln -sf /usr/share/zoneinfo/Asia/Seoul /etc/localtime
  - sudo date -s " 2025-12-10 10:30:00"
  - date
- **핫스팟 사용 시 (time server 동기화)**
  - sudo apt install rdate -y
  - sudo rdate -s time.bora.net
  - sudo ln -sf /usr/share/zoneinfo/Asia/Seoul /etc/localtime
  - date

```
pi@raspberrypi:~/controller $ date
Wed 10 Dec 17:46:21 KST 2025
pi@raspberrypi:~/controller $
```

Confidential



가상환경을 만들고 필요한 패키지를 설치한다.

- python3 -m venv venv
- source venv/bin/activate

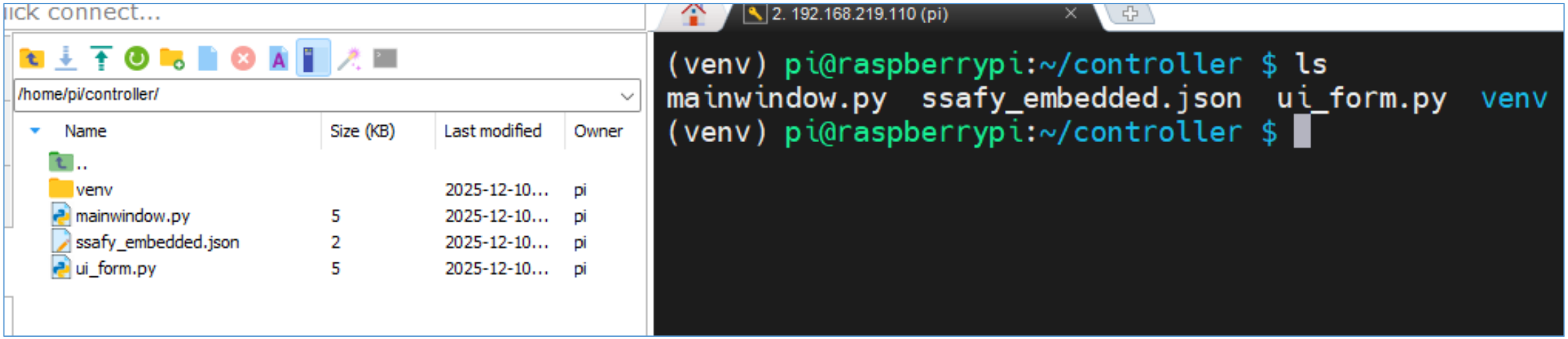
```
pi@raspberrypi:~/controller $ ls
pi@raspberrypi:~/controller $ python3 -m venv venv
pi@raspberrypi:~/controller $ source venv/bin/activate
```

- pip install firebase-admin pytz PySide6

```
(venv) pi@raspberrypi:~/controller $ pip install firebase-admin pytz
Looking in indexes: https://pypi.org/simple, https://www.piwheels.org/simple
Collecting firebase-admin
  Downloading https://www.piwheels.org/simple/firebase-admin/firebase_admin-7.1.
0-py3-none-any.whl (137 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 137.1/137.1 kB 219.8 kB/s eta 0:00:00
Requirement already satisfied: pytz in ./venv/lib/python3.11/site-packages (2025
```

GUI App 파일을 복사한다.

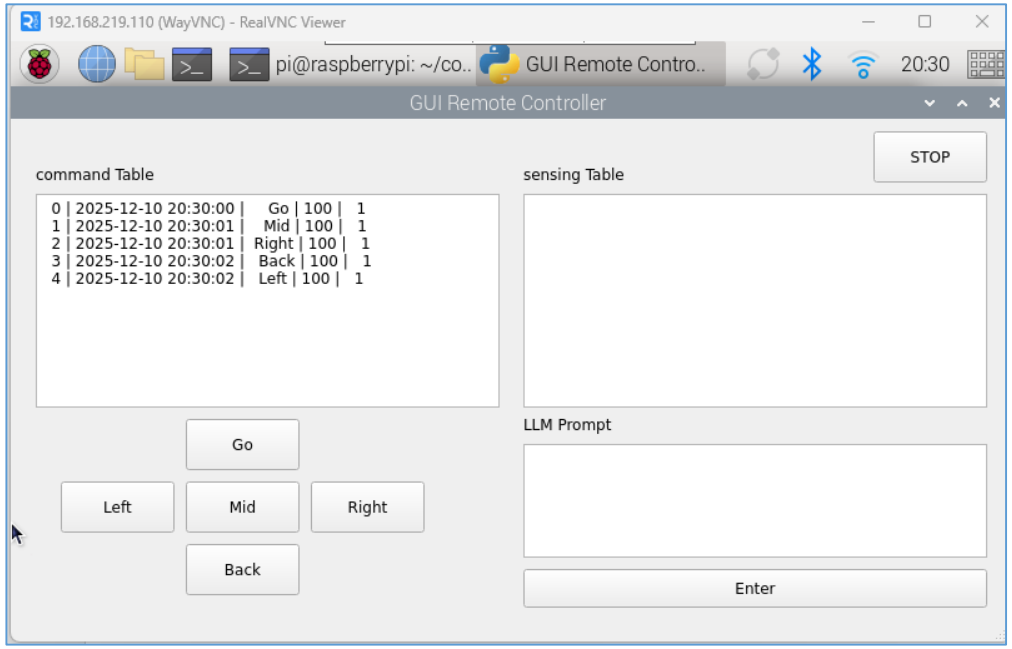
- mainwindow.py
- ui_form.py
- ssafy_embedded.json

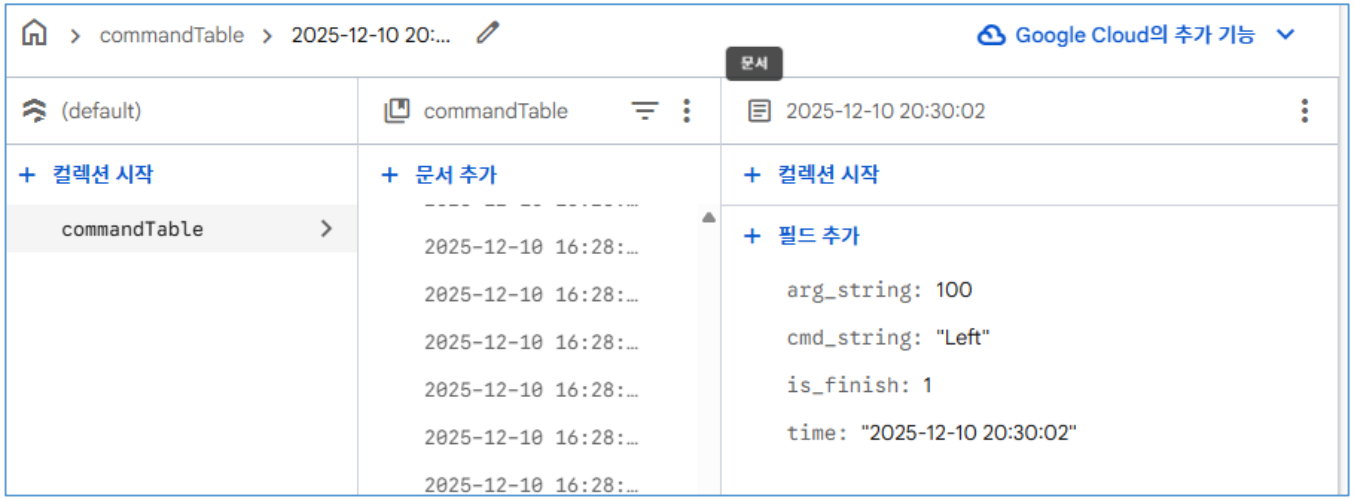


Confidential

라즈베리파이에서도 성공!

- python3 mainwindow.py
- 가상환경 PySide6 설치하여 동작





SAMSUNG SW AI ACADEMY FOR YOUTH

함께가요 미래로!
Enabling People

내일 방송에서 만나요!

삼성청년SW·AI아카데미